PROPOSAL TUGAS AKHIR - EE234799

IMPLEMENTASI PERANGKAT KERAS ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

KEINDRA DANTE HIDAYAT

NRP 5022231236

Dosen Pembimbing

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Program Studi Sarjana 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

PROPOSAL TUGAS AKHIR - EE234799

IMPLEMENTASI PERANGKAT KERAS ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

KEINDRA DANTE HIDAYAT

NRP 5022231236

Dosen Pembimbing

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Program Studi Sarjana 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

FINAL PROJECT PROPOSAL - EE234799

HARDWARE IMPLEMENTATION OF FXLMS AND FXNLMS ALGORITHMS IN AN FPGA-BASED ACTIVE NOISE CANCELLATION (ANC) HEADPHONE SYSTEM

KEINDRA DANTE HIDAYAT

NRP 5022231236

Advisor

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Undergraduate Study Program of Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

ABSTRAK

Nama Mahasiswa : KEINDRA DANTE HIDAYAT

Judul Tugas Akhir : IMPLEMENTASI PERANGKAT KERAS ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

Pembimbing : 1. Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

Active Noise Cancellation (ANC) digunakan untuk meredam kebisingan frekuensi rendah yang tidak dapat diatasi secara efektif oleh peredam pasif. Algoritma *Filtered-X Least Mean Square* (FXLMS) menjadi standar pada sistem ANC, tetapi *step-size* yang tetap membuat kinerjanya bergantung pada *tuning* manual dan rentan terhadap variasi daya sinyal kebisingan *broadband*. *Filtered-X Normalized Least Mean Square* (FXNLMS) mengatasi kelemahan tersebut dengan menormalisasi *step-size* terhadap daya sinyal referensi terfilter, tetapi menambah estimasi daya dan operasi pembagian pada setiap iterasi. Penelitian terdahulu umumnya hanya menyoroti kinerja peredaman atau efisiensi perangkat keras secara terpisah, sehingga belum diketahui apakah keunggulan FXNLMS sepadan dengan biaya komputasi tambahannya. Penelitian ini membandingkan FXLMS dan FXNLMS pada sistem ANC *headphone feedforward single-channel* (SISO) melalui tiga tahap, yaitu simulasi kinerja peredaman menggunakan *white noise*, *pink noise*, dan rekaman bising lingkungan nyata; analisis pengaruh normalisasi *step-size* terhadap kecepatan konvergensi dan *steady-state error*; serta implementasi *fixed-point* pada FPGA Artix-7 XC7A35T. Kinerja dievaluasi melalui peredaman kebisingan (SNR), kecepatan konvergensi, dan *steady-state error*, sedangkan efisiensi komputasi dievaluasi melalui kompleksitas operasi per-iterasi serta utilisasi LUT, *flip-flop*, memori, dan DSP *slice*. Hasil penelitian diharapkan memberikan dasar kuantitatif dalam memilih algoritma ANC untuk perangkat tersemat berkapasitas terbatas.

Kata Kunci: *Active Noise Cancellation*, FXLMS, FXNLMS, FPGA, Efisiensi Komputasi

ABSTRACT

Name : KEINDRA DANTE HIDAYAT
Title : *HARDWARE IMPLEMENTATION OF FXLMS AND FXNLMS ALGORITHMS IN AN FPGA-BASED ACTIVE NOISE CANCELLATION (ANC) HEADPHONE SYSTEM*
Advisors : 1. Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

Active Noise Cancellation (ANC) is used to attenuate low-frequency noise that passive attenuation cannot suppress. The Filtered-X Least Mean Square (FXLMS) algorithm is the standard in ANC systems, but its fixed step-size makes its performance dependent on manual tuning and sensitive to power variations in broadband noise. The Filtered-X Normalized Least Mean Square (FXNLMS) algorithm addresses this weakness by normalizing the step-size to the power of the filtered reference signal, but adds a power estimate and a division per iteration. Previous studies generally examined either noise reduction performance or hardware efficiency separately, so it remains unclear whether FXNLMS justifies its additional computational cost. This research compares FXLMS and FXNLMS in a feedforward single-channel (SISO) headphone ANC system in three stages: noise reduction simulation using white noise, pink noise, and real-world ambient noise recordings; analysis of how step-size normalization affects convergence speed and steady-state error; and fixed-point implementation on an Artix-7 XC7A35T FPGA. Performance is evaluated through noise reduction (SNR), convergence speed, and steady-state error, while computational efficiency is evaluated through per-iteration operation complexity and the utilization of LUTs, flip-flops, memory, and DSP slices. The results are expected to give a quantitative basis for selecting ANC algorithms for resource-constrained embedded devices.

Keywords: *Active Noise Cancellation*, *FXLMS*, *FXNLMS*, *FPGA*, *Computational Efficiency*

DAFTAR ISI

ABSTRAK i

ABSTRACT ii

DAFTAR ISI iii

DAFTAR GAMBAR v

DAFTAR TABEL vi

1 PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Rumusan Masalah 1

1.3 Batasan Masalah 2

1.4 Tujuan 2

1.5 Manfaat 3

1.5.1 Bagi Peneliti 3

1.5.2 Bagi Masyarakat Umum 3

2 TINJAUAN PUSTAKA 4

2.1 Hasil Penelitian Terdahulu 4

2.2 Dasar Teori 6

2.2.1 *Active Noise Cancellation* (ANC) 6

2.2.2 Filter Adaptif dan Algoritma LMS 8

2.2.3 Algoritma FXLMS 9

2.2.4 Algoritma FXNLMS 10

2.2.5 *Field-Programmable Gate Array* (FPGA) 11

3 METODOLOGI 13

3.1 Metode Penelitian 13

3.2 Alat dan Bahan Penelitian 13

3.3 Alur Pelaksanaan Penelitian 15

3.4 Perancangan Sistem ANC *Headphone* Berbasis FPGA 17

3.4.1 Arsitektur Sistem 17

3.4.2 Parameter Rancangan 20
3.4.3 Model Jalur Akustik dan Penetapan *Step-Size* 20
3.4.4 Estimasi Jalur Sekunder 21
3.4.5 Implementasi dan Perakitan FXLMS dan FXNLMS 22
3.4.6 Arsitektur Perangkat Keras pada FPGA 24
3.5 Metode Pengujian dan Analisis Data 25
3.5.1 Verifikasi Fungsional Rancangan RTL 25
3.5.2 Pengujian Kinerja Peredaman 26
3.5.3 Pengujian Kecepatan Konvergensi dan *Steady-State Error* 26
3.5.4 Evaluasi Kebutuhan Sumber Daya Perangkat Keras 27
3.5.5 Analisis Data Akhir 28

DAFTAR PUSTAKA **29**

DAFTAR GAMBAR

3.1 Diagram alir pelaksanaan penelitian 16
3.2 Diagram blok sistem simulasi ANC *headphone* 18
3.3 Diagram blok prototipe ANC *headphone* berbasis FPGA 19
3.4 Skema estimasi jalur sekunder secara *offline* 21
3.5 Alur pemrosesan setiap sampel pada FXLMS dan FXNLMS 23

DAFTAR TABEL

2.1 Hasil Penelitian Terdahulu 4
2.2 Perbandingan FXLMS dan FXNLMS 11

3.1 Alat dan bahan penelitian 13
3.2 Repositori sinyal bising lingkungan nyata 15
3.3 Jadwal pelaksanaan tugas akhir 17

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Polusi suara menurunkan kenyamanan akustik di berbagai lingkungan, mulai dari kabin kendaraan hingga perangkat audio portabel. *Passive Noise Cancellation* (PNC) efektif meredam kebisingan frekuensi tinggi, tetapi kinerjanya menurun drastis pada frekuensi rendah [1, 2]. *Active Noise Cancellation* (ANC) mengatasi keterbatasan tersebut dengan meredam kebisingan melalui prinsip interferensi destruktif secara langsung.

Algoritma *Filtered-X Least Mean Square* (FXLMS) menjadi standar pada ANC karena mampu mengompensasi jalur sekunder dan menjaga stabilitas konvergensi [2, 3]. Namun, FXLMS menggunakan *step-size* (μ) tetap sehingga kinerjanya bergantung pada *tuning* manual dan rentan terhadap perubahan daya kebisingan frekuensi rendah [4]. *Filtered-X Normalized Least Mean Square* (FXNLMS) mengatasi kelemahan ini dengan menormalisasi *step-size* terhadap energi sinyal referensi terfilter dan dilaporkan berkonvergensi lebih cepat daripada FXLMS standar [4]. Pada *headphone*, kedua algoritma juga harus memenuhi kendala kausalitas ANC *feedforward*, yaitu pencuplikan, perhitungan, dan pembangkitan *anti-noise* harus selesai sebelum *noise* merambat dari mikrofon referensi ke mikrofon *error* [5], dengan jendela waktu yang sangat pendek [6]. FPGA sesuai untuk kebutuhan ini karena operasi *multiply-accumulate* (MAC) yang mendominasi FXLMS dapat dipetakan langsung ke blok perangkat keras khusus dengan *delay* pemrosesan yang kecil [7]. Di sisi lain, FXNLMS menambahkan perhitungan energi sinyal dan operasi pembagian pada setiap iterasi [8], sedangkan blok khusus pada FPGA umumnya berupa memori dan pengali [9], sehingga pembagi harus dibangun dari sumber daya logika lain dan berpotensi menambah utilisasi serta latensi.

Dengan demikian, penerapan FXNLMS pada FPGA melibatkan *trade-off*: normalisasi *step-size* berpotensi mempercepat konvergensi, tetapi menuntut sumber daya perangkat keras tambahan. Selain itu, perhitungan energi sinyal dan operasi pembagian dalam aritmetika *fixed-point* memiliki presisi yang terbatas, sehingga keunggulan konvergensi FXNLMS belum tentu dapat dipertahankan sepenuhnya setelah diwujudkan dalam perangkat keras. Untuk menentukan algoritma yang lebih tepat bagi ANC *headphone* berbasis FPGA, *trade-off* tersebut perlu diukur secara kuantitatif pada kondisi implementasi yang sama.

Oleh karena itu, penelitian ini mengimplementasikan FXLMS dan FXNLMS sebagai perangkat keras digital pada platform FPGA dan arsitektur yang sama untuk sistem ANC *headphone*. Kedua implementasi kemudian dibandingkan dari tiga sisi, yaitu kemampuan meredam kebisingan frekuensi rendah, perilaku konvergensi ketika daya kebisingan berfluktuasi, dan tambahan sumber daya perangkat keras yang dibutuhkan FXNLMS.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan algoritma FXLMS dan FXNLMS sebagai perangkat keras digital pada FPGA untuk sistem ANC *headphone*?
2. Bagaimana perbandingan kinerja peredaman kebisingan frekuensi rendah antara imple-

mentasi FXLMS dan FXNLMS pada prototipe ANC *headphone*?

3. Bagaimana pengaruh normalisasi *step-size* pada FXNLMS terhadap kecepatan konvergensi dan *steady-state error* saat daya kebisingan frekuensi rendah berfluktuasi, dibandingkan dengan FXLMS yang menggunakan *step-size* tetap?

4. Berapa besar tambahan sumber daya perangkat keras yang dibutuhkan FXNLMS dibandingkan FXLMS, ditinjau dari jumlah operasi per-iterasi, utilisasi sumber daya FPGA, dan latensi pemrosesan per sampel?

1.3 Batasan Masalah

Batasan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Topologi Sistem: Sistem ANC dibatasi pada topologi *feedforward single-channel* (SISO, *Single-Input Single-Output*) untuk satu sisi *earcup* pada *headphone*.

2. Cakupan Algoritma: Algoritma yang diimplementasikan dan dibandingkan hanya FXLMS standar dengan *step-size* tetap dan FXNLMS dengan *step-size* ternormalisasi. Jalur sekunder diestimasi secara *offline* sebelum sistem dijalankan dan tidak diperbarui selama ANC bekerja.

3. Platform Implementasi: Kedua algoritma diimplementasikan menggunakan HDL dengan aritmetika *fixed-point* pada platform FPGA yang sama, dengan arsitektur, panjang filter, lebar bit, frekuensi *clock*, dan frekuensi *sampling* yang sama.

4. Sinyal Kebisingan: Sinyal kebisingan yang digunakan hanya rekaman bising lingkungan nyata dari repositori DEMAND, QUT-NOISE, dan MUSAN pada pita frekuensi rendah 100–1000 Hz.

5. Metrik Evaluasi: Kinerja implementasi dievaluasi melalui besar peredaman kebisingan (*noise reduction*, NR) dalam dB, kecepatan konvergensi, dan *steady-state error* yang diukur pada prototipe, dengan hasil simulasi sebagai model acuan. Kebutuhan sumber daya perangkat keras dievaluasi melalui jumlah operasi per-iterasi (perkalian, penjumlahan, dan pembagian), utilisasi sumber daya FPGA (LUT, *flip-flop*, BRAM, dan DSP *slice*), serta latensi pemrosesan per sampel.

1.4 Tujuan

Berdasarkan rumusan masalah, tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan algoritma FXLMS dan FXNLMS sebagai perangkat keras digital pada FPGA yang bekerja pada prototipe ANC *headphone*.

2. Membandingkan besar peredaman kebisingan frekuensi rendah antara implementasi FXLMS dan FXNLMS pada prototipe ANC *headphone*.

3. Menganalisis pengaruh normalisasi *step-size* pada FXNLMS terhadap kecepatan konvergensi dan *steady-state error* pada kondisi daya kebisingan tetap dan berfluktuasi, dibandingkan dengan FXLMS.

4. Mengukur dan membandingkan kebutuhan sumber daya perangkat keras FXLMS dan FXNLMS berdasarkan jumlah operasi per-iterasi, utilisasi sumber daya FPGA, dan latensi pemrosesan per sampel.

1.5 Manfaat

Manfaat yang diharapkan dari penelitian ini adalah sebagai berikut.

1.5.1 Bagi Peneliti

1. Meningkatkan pemahaman dan keterampilan dalam perancangan sistem ANC, khususnya algoritma filter adaptif FXLMS dan FXNLMS, aritmetika *fixed-point*, serta perancangan, implementasi, dan verifikasi perangkat keras digital pada FPGA.

2. Menerapkan pengetahuan yang diperoleh selama pendidikan sarjana pada bidang pengolahan sinyal digital, sistem digital, dan rekayasa elektronika.

1.5.2 Bagi Masyarakat Umum

1. Menyediakan rancangan acuan perangkat keras FXLMS dan FXNLMS pada FPGA untuk sistem ANC *headphone*, beserta data kuantitatif mengenai kinerja peredaman dan kebutuhan sumber daya perangkat kerasnya, sebagai dasar dalam memilih dan mengimplementasikan algoritma ANC pada perangkat tersemat berkapasitas terbatas.

BAB 2 TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Penelitian mengenai sistem ANC berbasis algoritma keluarga LMS telah dilakukan dengan objek, metode, dan platform implementasi yang beragam. Untuk memetakan posisi penelitian ini, lima penelitian terdahulu yang paling relevan dirangkum pada Tabel 2.1 berdasarkan objek atau data yang digunakan, metode, parameter dan hasil utama, serta keterkaitannya dengan penelitian ini.

Tabel 2.1: Hasil Penelitian Terdahulu

No.	Peneliti	Objek/Data	Metode	Parameter dan Hasil Utama	Keterikatan dengan Penelitian
1.	Xie, Zhou & Xie (2022)	Derau acak (*random source*, f_s 60 kHz) pada model *feedforward* untuk *earphone* ANC	Simulasi FxLMS di MATLAB/Simulink, ditambah *low-pass filter* (LPF) FIR/Chebyshev orde 16 pada sinyal residu	Orde filter 5-30 dan *step size* 0,0001-1. Pada orde 8 dan μ = 0,5, STD derau 0,04797 turun menjadi 0,000778 (0,0003826 dengan LPF). Orde terlalu rendah tidak konvergen, sedangkan orde dan μ besar membuat sistem divergen	Menjadi acuan pengaruh orde filter dan *step size* terhadap kecepatan konvergensi dan stabilitas FxLMS pada sistem *headphone*. Masih berupa simulasi, implementasi *hardware* disebut sebagai penelitian lanjutan
2.	Han & Chen (2023)	Nada tunggal 200 Hz dan 300 Hz (±55 dB), serta perpindahan 200 Hz ke 300 Hz	*Hybrid fixed-filter* + FxLMS: FFT memilih koefisien pra-latih sebagai nilai awal FxLMS. Diimplementasikan pada mikrokontroler STM32H743	Orde 128, f_s 20 kHz, μ = 0,001. Reduksi ±15 dB (200 Hz) tidak berubah, tetapi waktu konvergensi 300 Hz turun dari 10 s menjadi 5 s. Pada simulasi (μ = 0,005), konvergensi dari ±50.000 sampel menjadi ±20.000 sampel	Menunjukkan kelemahan FxLMS konvensional (konvergensi lambat, beban komputasi) pada sistem digital nyata. Orde 128 dan f_s 20 kHz dapat menjadi pembanding parameter. Tidak membandingkan dengan NLMS dan tidak melaporkan penggunaan sumber daya

Tabel 2.1: Hasil Penelitian Terdahulu (lanjutan)

No.	Peneliti	Objek/Data	Metode	Parameter dan Hasil Utama	Keterikatan dengan Penelitian
3.	Voltolini et al. (2023)	Derau struktural pembangkit kogenerasi; 3 akselerometer sebagai referensi, 3 *shaker*, 1 mikrofon *error* pada jarak 40 m	FxNLMS *offline* di MATLAB (SISO hingga SIMO multi-referensi), lalu uji *real-time hardware-in-the-loop* pada DSP ADAU1466Z	Rentang 50-250 Hz, orde W = 256, *downsampling* 16× (48 kHz ke 3 kHz), μ = 0,1. Reduksi 3,6-4,0 dB(A) (terbaik J = 2, K = 2: 4,0 dB(A)). Hasil *real-time* identik dengan *offline*, konvergensi ±7 s	Salah satu dari sedikit *paper* yang menjalankan FxNLMS pada perangkat keras khusus secara *real-time*. Teknik *downsampling* untuk menekan beban komputasi relevan bagi FPGA. Bukan aplikasi *headphone*, tanpa pembanding FxLMS dan tanpa laporan sumber daya DSP
4.	Chen, Yang, Lai & Wei (2025)	Derau impulsif frekuensi rendah (tembakan artileri) untuk perlindungan pendengaran	FxLMS arsitektur *feedback* (AANC) + *audio-balance control circuit* (ABCC); simulasi MATLAB R2023b dan implementasi rangkaian elektronik	Simulasi orde 30-60, μ antara 10^{-3} dan 10^{-2}. Konfigurasi terbaik pada *hardware* adalah orde 30 dan $\mu = 10^{-3}$. Reduksi maksimum 21,8 dB, dan 10-20 dB di bawah 2 kHz	Menegaskan bahwa orde dan *step size* harus di-*tuning* ulang saat berpindah dari simulasi ke *hardware*. Berbasis rangkaian elektronik (op-amp TL074/OPA810), bukan FPGA. Tidak ada FxNLMS dan tidak ada analisis sumber daya digital
5.	Cesur et al. (2025)	Derau jalan di kabin kendaraan listrik (rekaman lintasan uji, 2 mikrofon di posisi kepala pengemudi)	FxNLMS dengan *variable step-size*, pemodelan jalur sekunder *online* (PRBS + NLMS), *high-pass filter* dan *soft limiter*	f_s 44,1 kHz, *buffer* 256 sampel (±5,8 ms). Reduksi rata-rata 5,07 dB (8,28 dB di 250 Hz, 7,31 dB di 750 Hz), turun menjadi 2,4-3,6 dB di atas 1 kHz. Dibanding FxLMS standar: konvergensi lebih cepat dan reduksi lebih besar (disebut kualitatif, tanpa angka)	Paling dekat dengan topik penelitian ini karena menyatakan FxNLMS lebih unggul dari FxLMS dan cocok untuk sistem *embedded* dengan daya komputasi terbatas. Perbandingannya tidak kuantitatif, bukan di FPGA, dan tidak ada laporan sumber daya

Dua penelitian pertama mengkaji FXLMS pada tingkat simulasi dan mikrokontroler. Xie et al. [3] mensimulasikan FXLMS pada model *feedforward* untuk *earphone* ANC di MATLAB/Simulink dan menunjukkan bahwa orde filter dan *step-size* menentukan kestabilan algoritma. Pada orde 8 dan $\mu = 0{,}5$, standar deviasi derau turun dari 0,04797 menjadi 0,000778, sedangkan orde yang terlalu rendah gagal konvergen dan kombinasi orde serta μ yang besar membuat sistem divergen. Han and Chen [2] membawa FXLMS ke mikrokontroler STM32H743 dengan pendekatan *hybrid fixed-filter*. Reduksi sekitar 15 dB pada 200 Hz tidak berubah, tetapi waktu konvergensi pada 300 Hz berkurang dari 10 s menjadi 5 s. Kedua penelitian ini menegaskan bahwa FXLMS sensitif terhadap pemilihan μ dan cenderung lambat berkonvergensi, tetapi keduanya belum membandingkan FXLMS dengan varian

ternormalisasinya dan tidak melaporkan penggunaan sumber daya perangkat keras.

Dari sisi implementasi perangkat keras, Chen et al. [1] merealisasikan FXLMS dengan arsitektur *feedback* dalam bentuk rangkaian elektronik untuk derau impulsif frekuensi rendah dan memperoleh reduksi maksimum 21,8 dB. Penelitian tersebut juga menunjukkan bahwa rentang parameter pada simulasi, yaitu orde 30–60 dan μ antara 10^{-3} dan 10^{-2}, harus disesuaikan kembali menjadi orde 30 dan $\mu = 10^{-3}$ ketika dipindahkan ke *hardware*. Sementara itu, Voltolini et al. [10] menjalankan FXNLMS secara *real-time* pada DSP ADAU1466Z untuk derau struktural pembangkit kogenerasi dan memperoleh hasil yang identik dengan simulasi *offline*, dengan reduksi 3,6–4,0 dB(A). Teknik *downsampling* 16× yang digunakan untuk menekan beban komputasi pada penelitian tersebut relevan bagi implementasi FPGA. Meskipun demikian, kedua penelitian ini tidak menyandingkan FXLMS dan FXNLMS secara langsung. Penelitian pertama tidak menggunakan FXNLMS, penelitian kedua tidak menggunakan FXLMS sebagai pembanding, dan keduanya tidak menyertakan analisis sumber daya digital.

Penelitian yang paling dekat dengan topik ini adalah Cesur et al. [4], yang menerapkan FXNLMS dengan *variable step-size* dan pemodelan jalur sekunder *online* pada kabin kendaraan listrik. Penelitian tersebut memperoleh reduksi rata-rata 5,07 dB, dengan reduksi terbesar 8,28 dB pada 250 Hz, dan reduksi yang turun menjadi 2,4–3,6 dB di atas 1 kHz. FXNLMS dinyatakan lebih cepat berkonvergensi dan menghasilkan reduksi yang lebih besar dibandingkan FXLMS standar, tetapi perbandingan tersebut hanya disampaikan secara kualitatif tanpa data kuantitatif, tidak diimplementasikan pada FPGA, dan tidak disertai laporan penggunaan sumber daya.

2.2 Dasar Teori

2.2.1 *Active Noise Cancellation* (ANC)

Active Noise Cancellation (ANC) bekerja dengan membangkitkan gelombang suara dari sumber sekunder yang memiliki amplitudo sama dan fasa berlawanan terhadap sumber suara primer, sehingga keduanya saling meniadakan melalui interferensi destruktif [11]. Gelombang *anti-noise* yang berbeda fasa 180° terhadap *noise* akan meredam *noise* tersebut secara efektif [6]. Kebisingan frekuensi rendah sulit dikendalikan dengan solusi peredaman pasif konvensional [1, 6].

Prinsip tersebut dapat dinyatakan melalui superposisi dua gelombang tekanan. Gelombang primer dan gelombang sekunder dinyatakan sebagai [11]:

$$p_1 = A\cos(\omega t - \varphi) \tag{2.1}$$

$$p_2 = \beta A\cos(\omega t - \varphi + \alpha) \tag{2.2}$$

dengan:

- p_1 : tekanan akustik gelombang primer (*noise*),
- p_2 : tekanan akustik gelombang sekunder (*anti-noise*),
- A : amplitudo gelombang primer,
- ω : frekuensi sudut gelombang (rad/s),
- t : waktu (s),
- φ : fasa awal gelombang primer (rad),

- β : perbandingan amplitudo gelombang sekunder terhadap gelombang primer,
- α : selisih fasa gelombang sekunder terhadap gelombang primer (rad).

Besar peredaman energi akibat superposisi kedua gelombang adalah [11]:

$$\Delta = 10 \lg \frac{E_1}{E_2} = -10 \lg \left(1 + 2\beta \cos \alpha + \beta^2\right) \tag{2.3}$$

dengan:

- Δ : besar peredaman energi (dB),
- E_1 : energi gelombang primer sebelum superposisi,
- E_2 : energi hasil superposisi gelombang primer dan gelombang sekunder,
- lg : logaritma basis 10,
- β, α : perbandingan amplitudo dan selisih fasa seperti pada Persamaan (2.2).

Pada sistem ANC digital, sinyal residual yang terukur pada mikrofon *error* merupakan jumlah dari *noise* primer dan keluaran filter kontrol setelah melewati jalur sekunder [11, 12]:

$$e(n) = d(n) + \sum_{i=0}^{M-1} s_i \, y(n - i) \tag{2.4}$$

dengan:

- n : indeks sampel waktu diskrit,
- $e(n)$: sinyal residual (*error*) yang terukur pada mikrofon *error*,
- $d(n)$: *noise* primer pada posisi mikrofon *error*,
- s_i : koefisien ke-i respons impuls jalur sekunder,
- M : panjang respons impuls jalur sekunder,
- $y(n - i)$: keluaran filter kontrol pada sampel ke-$(n - i)$.

Berdasarkan sumber sinyal yang digunakan untuk membentuk sinyal kontrol, sistem ANC dibedakan menjadi beberapa topologi. Pada sistem ANC *feedforward*, sinyal referensi diperoleh dari sumber *noise*. Pada sistem ANC *feedback*, sinyal kontrol diperoleh hanya dengan memfilter sinyal *error* [11], sehingga arsitektur *feedback* tidak memiliki mikrofon referensi tambahan untuk mencuplik sinyal eksternal [1]. Selain kedua topologi tersebut, terdapat pula topologi *hybrid* [1, 11].

Sistem ANC *feedforward* pada *headset* tersusun atas mikrofon referensi eksternal, mikrofon *error* internal, dan sumber sekunder di dalam *earmuff*. Algoritma *filtered-x least mean square* (FXLMS) digunakan untuk mengadaptasi filter digital yang menggerakkan sumber sekunder tersebut [5].

Sistem ANC *feedforward* memiliki kendala kausalitas, yaitu *delay* jalur primer harus lebih besar daripada total *delay* jalur sekunder. Jika kendala ini tidak terpenuhi, *bandwidth* peredaman menyempit dan peredaman maksimum menurun [5]. Pada perangkat berukuran kecil, *delay* akustik antara mikrofon referensi dan mikrofon *error* menjadi lebih pendek sehingga jendela waktu pemrosesan yang tersedia semakin terbatas. Oleh karena itu, prosesor harus menyelesaikan pencuplikan sinyal referensi, pemrosesan, dan pembangkitan *anti-noise* dengan cepat [6].

2.2.2 Filter Adaptif dan Algoritma LMS

Pada sistem ANC, filter adaptif dapat berbentuk filter *Finite Impulse Response* (FIR), yaitu filter dengan struktur *all-zero* [1]. Keluaran filter FIR adaptif dengan N koefisien dinyatakan sebagai [11]:

$$y(n) = \mathbf{W}^T(n)\,\mathbf{X}(n) = \sum_{i=0}^{N-1} w_i(n)\,x(n-i) \tag{2.5}$$

dengan:

- $y(n)$: keluaran filter adaptif pada sampel ke-n,
- $\mathbf{W}(n) = [w_0(n), w_1(n), \ldots, w_{N-1}(n)]^T$: vektor koefisien filter pada sampel ke-n,
- $w_i(n)$: koefisien ke-i filter pada sampel ke-n,
- $\mathbf{X}(n) = [x(n), x(n-1), \ldots, x(n-N+1)]^T$: vektor N sampel masukan terbaru,
- $x(n-i)$: sampel sinyal masukan pada sampel ke-$(n-i)$,
- N : jumlah koefisien (panjang) filter,
- $(\cdot)^T$: operasi *transpose*.

Galat antara sinyal yang diinginkan dan keluaran filter adalah [11]:

$$e(n) = d(n) - y(n) \tag{2.6}$$

dengan:

- $e(n)$: sinyal galat pada sampel ke-n,
- $d(n)$: sinyal yang diinginkan (*desired signal*) pada sampel ke-n,
- $y(n)$: keluaran filter adaptif pada Persamaan (2.5).

Tujuan adaptasi adalah meminimalkan fungsi biaya *Mean Square Error* (MSE) [11]:

$$J(n) = E\left[e^2(n)\right] \tag{2.7}$$

dengan:

- $J(n)$: fungsi biaya MSE pada sampel ke-n,
- $E[\cdot]$: operator nilai harapan (*expected value*),
- $e(n)$: sinyal galat pada Persamaan (2.6).

Metode *steepest descent* memperbarui koefisien filter berlawanan arah dengan gradien fungsi biaya [11]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) - \mu\,\nabla(n) \tag{2.8}$$

dengan:

- $\mathbf{W}(n+1)$: vektor koefisien filter hasil pembaruan untuk sampel berikutnya,
- μ : *step-size*, yaitu konstanta positif yang mengatur besar langkah pembaruan,
- $\nabla(n)$: gradien fungsi biaya $J(n)$ terhadap vektor koefisien $\mathbf{W}(n)$.

Algoritma *Least Mean Square* (LMS) menggunakan estimasi gradien sesaat $\hat{\nabla}$ =

$-2e(n)\mathbf{X}(n)$, yaitu gradien dari $e^2(n)$ sebagai pengganti nilai harapan $E[e^2(n)]$, sehingga persamaan pembaruan koefisien LMS menjadi [11]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) + 2\mu\, e(n)\, \mathbf{X}(n) \tag{2.9}$$

dengan:

- $\hat{\nabla}$: estimasi gradien sesaat,
- μ : *step-size*,
- $e(n)$: sinyal galat pada Persamaan (2.6),
- $\mathbf{X}(n)$: vektor sampel masukan pada Persamaan (2.5).

Syarat konvergensi algoritma LMS dinyatakan sebagai [11]:

$$0 < \mu < \frac{1}{\text{tr}[\mathbf{R}]} \tag{2.10}$$

dengan:

- $\mathbf{R} = E[\mathbf{X}(n)\mathbf{X}^T(n)]$: matriks autokorelasi sinyal masukan,
- tr[$\cdot$] : *trace* matriks, yaitu jumlah elemen diagonal utama matriks.

Step-size menentukan kecepatan konvergensi dan kestabilan sistem [1, 11]. *Step-size* yang kecil menghasilkan konvergensi yang lambat, tetapi menghasilkan residual *error* kondisi tunak yang lebih rendah [8]. Hal ini menunjukkan adanya *trade-off* antara laju konvergensi dan kinerja kondisi tunak [12]. Sebaliknya, *step-size* yang terlalu besar dapat membuat sistem tidak stabil [1].

2.2.3 Algoritma FXLMS

Algoritma yang paling banyak digunakan pada aplikasi ANC adalah *filtered-x least mean square* (FXLMS) [13]. Pada sistem ANC, sinyal kontrol tidak langsung bertemu dengan *noise* pada mikrofon *error*, tetapi melewati jalur sekunder, yaitu respons impuls dari *speaker* kontrol ke mikrofon *error* [1, 12]. FXLMS memperhitungkan latensi akustik pada respons antara sumber sekunder dan sensor *error* tersebut [6].

Pada FXLMS, sinyal referensi difilter terlebih dahulu menggunakan estimasi jalur sekunder sehingga diperoleh sinyal referensi terfilter [11, 12]:

$$x_f(n) = \sum_{i=0}^{M-1} \hat{s}_i\, x(n-i) \tag{2.11}$$

$$\mathbf{X}_f(n) = \Big[x_f(n), x_f(n-1), \ldots, x_f(n-N+1)\Big]^T \tag{2.12}$$

dengan:

- $x_f(n)$: sinyal referensi terfilter (*filtered-x*) pada sampel ke-n,
- $\hat{s}_i$: koefisien ke-i estimasi respons impuls jalur sekunder,
- M : panjang estimasi respons impuls jalur sekunder,
- $x(n-i)$: sampel sinyal referensi dari mikrofon referensi pada sampel ke-$(n-i)$,

- $\mathbf{X}_f(n)$: vektor N sampel sinyal referensi terfilter terbaru,
- N : panjang filter adaptif.

Dengan sinyal *error* yang didefinisikan sebagai jumlah *noise* primer dan keluaran jalur sekunder seperti pada Persamaan (2.4), pembaruan koefisien FXLMS dinyatakan sebagai [11]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) - 2\mu\, e(n)\, \mathbf{X}_f(n) \tag{2.13}$$

dengan:

- $\mathbf{W}(n)$: vektor koefisien filter adaptif pada sampel ke-n,
- μ : *step-size* yang bernilai tetap,
- $e(n)$: sinyal *error* pada Persamaan (2.4),
- $\mathbf{X}_f(n)$: vektor sinyal referensi terfilter pada Persamaan (2.12).

Tanda negatif pada Persamaan (2.13) mengikuti definisi sinyal *error* pada Persamaan (2.4) [11].

2.2.4 Algoritma FXNLMS

Algoritma *filtered-x normalized least mean square* (FXNLMS) banyak digunakan pada sistem ANC [14]. Pada FXNLMS, *step-size* dinormalisasi terhadap energi vektor sinyal referensi terfilter [8, 12]. Dengan konvensi sinyal *error* yang sama dengan Persamaan (2.4), pembaruan koefisien FXNLMS dinyatakan sebagai [12]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) - \mu\, e(n)\, \frac{\mathbf{X}_f(n)}{\varepsilon + \left\|\mathbf{X}_f(n)\right\|^2} \tag{2.14}$$

dengan:

- μ : *step-size* sebelum dinormalisasi,
- ε : konstanta positif kecil ($\varepsilon > 0$) yang mencegah terjadinya pembagian dengan nol,
- $\left\|\mathbf{X}_f(n)\right\|^2$: kuadrat norma Euclidean vektor sinyal referensi terfilter, yaitu jumlah kuadrat seluruh elemen $\mathbf{X}_f(n)$,
- $\mathbf{W}(n)$, $e(n)$, $\mathbf{X}_f(n)$: sama seperti pada Persamaan (2.13).

Pembaruan FXLMS ternormalisasi disebutkan lebih disukai pada kondisi yang berubah terhadap waktu [12]. Namun, untuk sinyal masukan siklostasioner, kinerja peredaman FXNLMS dipengaruhi oleh daya periodik sinyal masukan sehingga menghasilkan *steady-state mean square deviation* (MSD) yang lebih besar dibandingkan FXLMS [14].

Pada algoritma NLMS, faktor normalisasi juga dapat dihitung dari estimasi daya sinyal masukan yang diperbarui secara rekursif [15]:

$$r(n) = (1-\alpha)\, r(n-1) + \alpha\, x^2(n) \tag{2.15}$$

dengan:

- $r(n)$: estimasi daya sinyal masukan pada sampel ke-n,
- $r(n-1)$: estimasi daya sinyal masukan pada sampel sebelumnya,
- α : faktor peluruhan (*forgetting factor*) yang dapat disesuaikan dengan tingkat ketidak-

stasioneran sinyal masukan,
- $x(n)$: sampel sinyal masukan pada sampel ke-n.

Estimasi daya tersebut digunakan pada penguatan adaptasi [15]:

$$\mathbf{g}(n) = \frac{\mu\,\mathbf{X}(n)}{c_0 + N\,r(n)} \tag{2.16}$$

dengan:

- $\mathbf{g}(n)$: vektor penguatan adaptasi yang menggantikan suku $\mu\,\mathbf{X}(n)$ pada pembaruan koefisien,
- c_0 : konstanta positif kecil untuk menghindari pembagian dengan nol ketika sinyal masukan tidak ada,
- N : panjang filter adaptif,
- $r(n)$: estimasi daya sinyal masukan pada Persamaan (2.15),
- μ, $\mathbf{X}(n)$: *step-size* dan vektor sampel masukan.

Dari sisi komputasi, setiap iterasi FXNLMS membutuhkan $3L + M + 1$ operasi perkalian, satu operasi pembagian, dan $3L + M - 3$ operasi penjumlahan, dengan L adalah panjang vektor koefisien filter adaptif (setara dengan N pada persamaan di atas) dan M adalah panjang respons impuls jalur sekunder [8]. Perbandingan kedua algoritma berdasarkan persamaan dan literatur di atas dirangkum pada Tabel 2.2.

Tabel 2.2: Perbandingan FXLMS dan FXNLMS

Aspek	**FXLMS**	**FXNLMS**
Pembaruan koefisien	Persamaan (2.13) [11]	Persamaan (2.14) [12]
Step-size	Tetap, μ	$\mu/(\varepsilon + \|\|\mathbf{X}_f(n)\|\|^2)$, mengikuti energi sinyal referensi terfilter
Operasi pembagian per iterasi	Tidak ada	Satu [8]
Catatan dari literatur	Algoritma yang paling banyak digunakan pada ANC [13]	Lebih disukai pada kondisi yang berubah terhadap waktu [12]; MSD tunak lebih besar daripada FXLMS untuk masukan siklostasioner [14]

2.2.5 *Field-Programmable Gate Array* (FPGA)

Field-Programmable Gate Array (FPGA) adalah perangkat terfabrikasi yang dapat diprogram secara elektris untuk menjadi hampir semua jenis rangkaian atau sistem digital. FPGA tersusun atas larik blok logika yang dapat diprogram, yang dikelilingi oleh jaringan *routing* dan blok I/O yang juga dapat diprogram [9]. Pada FPGA berbasis SRAM, konfigurasi bersifat *volatile* sehingga harus dimuat ulang setiap kali FPGA dinyalakan [9, 16].

Sumber daya logika FPGA dibentuk oleh *lookup table* (LUT) dan *flip-flop*. Pada FPGA 7 series, empat LUT 6 masukan beserta delapan *flip-flop* dan logika *carry* membentuk satu *slice* [16]. Selain itu, FPGA dilengkapi blok khusus (*hard block*) seperti memori dan pengali, yang umumnya lebih unggul dalam luas area, kecepatan, dan konsumsi daya dibandingkan fungsi yang sama pada logika serbaguna [9]. Pada FPGA 7 series, blok khusus tersebut meliputi *block RAM dual-port* 36 Kb dan DSP *slice* yang terdiri atas pengali *two's complement* 25×18 bit dan akumulator 48 bit dengan kemampuan *pipelining* [16].

Pemrosesan algoritma FXLMS ditandai oleh banyaknya operasi *multiply-accumulate* (MAC) yang harus diselesaikan dalam setiap periode sampel. Daya komputasi menjadi kriteria terpenting untuk memungkinkan filter berorde tinggi yang dikombinasikan dengan frekuensi *sampling* yang tinggi [7]. Kesesuaian suatu teknologi implementasi untuk ANC terutama ditentukan oleh daya pemrosesan, disertai *delay* pemrosesan dan efisiensi energi, dan kinerja ANC juga dapat meningkat dengan tambahan daya komputasi [7].

Selain daya komputasi, sistem ANC membutuhkan *delay* pemrosesan yang kecil karena waktu yang tersedia untuk meredam *noise* secara aktif terbatas [7]. Keterbatasan waktu ini berkaitan langsung dengan kendala kausalitas ANC *feedforward* pada Subbab 2.2.1, yaitu total *delay* jalur sekunder harus lebih kecil daripada *delay* jalur primer [5]. Pada perangkat berukuran kecil seperti *headphone*, *delay* akustik antara mikrofon referensi dan mikrofon *error* lebih pendek, sehingga jendela waktu pemrosesan semakin terbatas [6].

FPGA memiliki beberapa karakteristik yang menjawab kebutuhan tersebut. Pertama, pada logika FPGA, sampel audio dari antarmuka A/D dapat langsung diproses oleh gerbang dan register, sehingga *overhead* manajemen pada prosesor serbaguna atau sistem operasi dapat dihindari dan *delay* pemrosesan menjadi lebih kecil. Kedua, FPGA lebih dipilih untuk algoritma aritmetika sederhana yang sesuai untuk dieksekusi secara paralel. Ketiga, jalur data (*datapath*) pada FPGA dapat disesuaikan pada tingkat *register-transfer level* (RTL), yaitu tingkat abstraksi yang lebih rendah daripada urutan instruksi perangkat lunak pada prosesor DSP, sehingga optimasi dapat dilakukan lebih jauh [7]. Keempat, operasi perkalian dan akumulasi yang mendominasi FXLMS dapat dipetakan ke DSP *slice* yang memang dirancang sebagai pengali dan akumulator khusus dengan dukungan *pipelining* [16], sehingga memperoleh keunggulan blok khusus dalam luas area, kecepatan, dan konsumsi daya [9].

Pada prosesor DSP, utilisasi perangkat keras aritmetika per siklus *clock* dapat menurun secara signifikan ketika pembaruan koefisien harus ditangani bersamaan dengan proses konvolusi [7]. Pada FXLMS dan FXNLMS, kedua proses tersebut dilakukan pada setiap sampel, yaitu perhitungan keluaran filter pada Persamaan (2.5) dan pembaruan koefisien pada Persamaan (2.13) dan (2.14). Klemd juga mengutip hasil penelitian yang menunjukkan bahwa rancangan FPGA yang diparalelkan pada frekuensi 100 MHz dapat melampaui prosesor DSP 1,5 GHz dengan percepatan 12,3 kali dan konsumsi daya sekitar 2,4 kali lebih rendah pada teknologi proses yang sama [7].

Keunggulan tersebut terlihat pada hasil penelitian Klemd. Implementasi FXLMS *feedback* pada FPGA mencapai *delay* pemrosesan yang pendek dan tidak bergantung pada panjang filter. Pada konfigurasi paling berat, yaitu 11 sensor *error*, 11 sumber sekunder, orde filter 2049, dan frekuensi *sampling* 48 kHz, *delay* yang dicapai jauh di bawah 1 μs dengan tambahan peredaman hingga 12 dB dibandingkan implementasi perangkat lunak acuan [7]. Arsitektur FXLMS pada FPGA juga dapat diparameterisasi untuk diarahkan pada kinerja tinggi maupun efisiensi sumber daya [13].

BAB 3 METODOLOGI

3.1 Metode Penelitian

Penelitian ini menggunakan metode rancang bangun (*design and build*) yang dilanjutkan dengan eksperimen kuantitatif. Produk yang dirancang adalah implementasi perangkat keras algoritma FXLMS dan FXNLMS pada FPGA Artix-7 XC7A35T untuk sistem ANC *headphone feedforward* SISO. Kedua algoritma terlebih dahulu dirancang dalam Verilog dan diverifikasi melalui simulasi, kemudian dijalankan secara langsung pada prototipe ANC *headphone* untuk diukur kinerja peredaman dan kebutuhan sumber daya perangkat kerasnya.

Kedua algoritma dibangun dengan arsitektur, parameter, sinyal uji, dan prosedur pengukuran yang identik, sehingga perbedaan hasil hanya berasal dari mekanisme pembaruan koefisien. Hasil pengukuran pada prototipe menjadi data utama penelitian, sedangkan hasil simulasi digunakan sebagai acuan.

3.2 Alat dan Bahan Penelitian

Alat dan bahan penelitian ditunjukkan pada Tabel 3.1. Alat penelitian mencakup perangkat keras prototipe ANC, perangkat ukur dan pendukung, serta perangkat lunak simulasi, sintesis, implementasi, dan analisis data. Bahan penelitian berupa rekaman bising lingkungan nyata.

Tabel 3.1: Alat dan bahan penelitian

Alat/Bahan	**Spesifikasi**	**Fungsi**
Board FPGA	Digilent Arty A7-35T, Xilinx Artix-7 XC7A35T, 20.800 LUT, 41.600 FF, 50 BRAM 36 Kb, 90 DSP48E1, *clock* 100 MHz, USB-JTAG/UART	Platform implementasi perangkat keras FXLMS dan FXNLMS, jalur pencatatan data ke komputer, serta sumber catu 3,3 V bagi mikrofon, DAC, dan penguat
Mikrofon referensi	INMP441, MEMS, antarmuka I2S, 24-bit, *omnidirectional*, 1 unit	Akuisisi sinyal kebisingan di luar *earcup*
Mikrofon *error*	INMP441, MEMS, antarmuka I2S, 24-bit, *omnidirectional*, 1 unit	Akuisisi sinyal residual di dalam *earcup*, dekat telinga
DAC audio	PCM5102A, antarmuka I2S, 32-bit, 112 dB SNR	Konversi sinyal *anti-noise* digital menjadi analog
Penguat *headphone*	TPA6132A2, stereo, *class*-G, 2,3–5,5 V	Penguatan sinyal analog untuk menggerakkan *speaker*
Speaker driver	Dinamis 40 mm, 32 Ω, 2 unit (1 aktif, 1 cadangan)	Pembangkit sinyal *anti-noise*
Earcup dan dudukan uji	*Earcup headphone over-ear* beserta bantalannya, dipasang pada dudukan uji dengan posisi mikrofon dan sumber kebisingan yang tetap	Rumah bagi *speaker driver* dan mikrofon *error*, serta menjaga posisi pengukuran tetap sama di setiap pengujian

Tabel 3.1: Alat dan bahan penelitian (lanjutan)

Alat/Bahan	**Spesifikasi**	**Fungsi**
Mikrofon ukur	MIC-L20 *Clip On Microphone* USB	Pengukuran independen tingkat peredaman di dalam *earcup*
Sumber kebisingan	*Speaker* aktif/*studio monitor*	Pemutar rekaman bising lingkungan nyata pada prototipe
Komputer	Windows 11/Linux	Menjalankan perangkat lunak penyiapan sinyal, simulasi, sintesis, implementasi, pemrograman FPGA, pencatatan data UART, dan analisis data
Vivado Design Suite	Perangkat lunak simulasi (xsim), sintesis, dan implementasi HDL untuk FPGA Xilinx, dijalankan melalui skrip Tcl	Simulasi RTL Verilog beserta *testbench*, estimasi jalur sekunder pada *testbench*, sintesis, implementasi, pembangkitan *bitstream*, pemrograman FPGA, serta pengambilan laporan utilisasi LUT, *flip-flop*, BRAM, DSP *slice*, dan *timing*
Python (NumPy, SciPy, Matplotlib)	Bahasa pemrograman beserta pustaka komputasi numerik, pengolahan sinyal, dan penampil grafik	Penyiapan sinyal uji, perhitungan metrik dari data simulasi dan prototipe, serta penampilan grafik
Rekaman bising lingkungan nyata	DEMAND, QUT-NOISE, dan MUSAN (Tabel 3.2)	Sinyal kebisingan untuk simulasi, estimasi jalur sekunder, penetapan *step-size*, verifikasi, dan pengujian prototipe

Seluruh sinyal kebisingan pada penelitian ini diambil dari tiga repositori *open-source* yang dirangkum pada Tabel 3.2. Ketiga repositori dipilih karena menyediakan bising dari lingkungan yang beragam, seperti kendaraan, jalan raya, kantor, dan ruang publik, dengan lisensi terbuka.

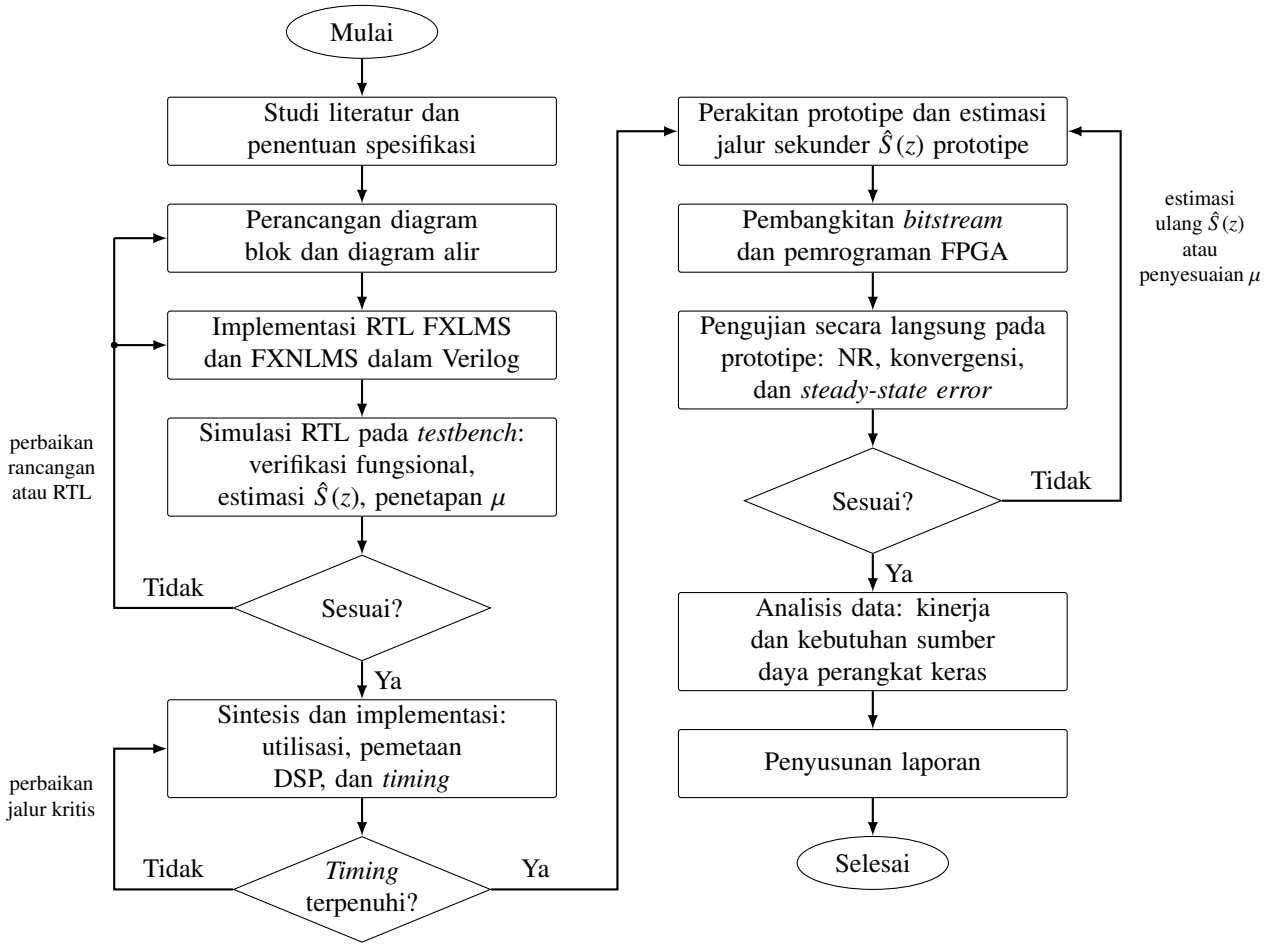
Tabel 3.2: Repositori sinyal bising lingkungan nyata

Dataset	**Repositori**	**Isi**	**Format**	**Lisensi**
DEMAND [17]	Zenodo (DOI 10.5281/ zenodo.1227121)	17 lingkungan, antara lain kendaraan (TBUS, TCAR, TMETRO), jalan raya (STRAFFIC), kantor, rumah, dan ruang publik	16 kanal, 48 kHz dan 16 kHz	CC BY 4.0
QUT-NOISE [18]	QUT Research Data Finder	20 sesi di 10 lokasi (CAFE, CAR, HOME, REVERB, STREET), lebih dari 10 jam, setiap sesi minimal 30 menit	Stereo, 48 kHz, 16 bit	CC BY-SA
MUSAN (bagian *noise*) [19]	OpenSLR (SLR17)	929 berkas, sekitar 6 jam; bising teknis dan bising lingkungan seperti mobil *idle*, angin, hujan, dan petir	16 kHz	CC BY 4.0

Sebelum digunakan, setiap rekaman diolah menggunakan Python menjadi satu kanal, difilter *band-pass* pada pita 100–1000 Hz sesuai batasan penelitian, dicuplik ulang ke 48 kHz, lalu disimpan sebagai berkas biner `.raw` 16 bit *signed*. Pencuplikan ulang diperlukan untuk rekaman MUSAN yang berformat 16 kHz. Rekaman yang sudah berformat 48 kHz hanya mengalami penyaringan dan konversi format. Berkas `.raw` dipilih karena dapat dibaca langsung oleh *testbench* Verilog. Berkas `.wav` hasil penyaringan yang sama digunakan sebagai sinyal yang diputar oleh sumber kebisingan pada pengujian prototipe.

3.3 Alur Pelaksanaan Penelitian

Alur pelaksanaan penelitian ditunjukkan pada Gambar 3.1. Penelitian diawali dengan studi literatur mengenai ANC, algoritma FXLMS dan FXNLMS, aritmetika *fixed-point*, serta implementasi filter adaptif pada FPGA. Dari studi tersebut ditentukan spesifikasi sistem, yaitu frekuensi *sampling*, frekuensi *clock*, panjang filter, dan lebar bit pada Subbab 3.4.2. Arsitektur kedua algoritma kemudian dirancang dalam bentuk diagram blok dan diagram alir, lalu diimplementasikan dalam Verilog.



Gambar 3.1: Diagram alir pelaksanaan penelitian

Rancangan RTL kemudian disimulasikan dalam lingkar ANC tertutup pada *testbench*. Pada tahap ini, jalur sekunder diestimasi terhadap model $S(z)$, *step-size* kedua algoritma ditetapkan melalui penelusuran beberapa nilai, dan fungsi rancangan diverifikasi. Hasil simulasi dinyatakan sesuai jika kriteria verifikasi pada Subbab 3.5.1 terpenuhi, yaitu setiap blok filter menghasilkan keluaran yang benar, kedua algoritma konvergen pada seluruh rekaman uji, dan tidak terjadi *overflow*. Jika belum sesuai, rancangan diagram blok atau RTL diperbaiki, kemudian simulasi diulang.

Rancangan yang sesuai disintesis dan diimplementasikan menggunakan Vivado untuk memperoleh utilisasi sumber daya dan *timing*. Jika *timing* pada frekuensi *clock* 100 MHz belum terpenuhi, jalur kritis diperbaiki, misalnya dengan menambah tahap *pipeline*. Prototipe kemudian dirakit dan jalur sekundernya diestimasi. Karena koefisien $\hat{S}(z)$ disimpan sebagai nilai awal BRAM, *bitstream* kedua algoritma dibangkitkan setelah $\hat{S}(z)$ prototipe diperoleh, lalu diprogram ke FPGA dan diuji secara langsung pada prototipe. Hasil pengujian prototipe dinyatakan sesuai jika kedua algoritma konvergen tanpa divergensi. Jika belum sesuai, $\hat{S}(z)$ prototipe diestimasi ulang atau μ ditelusuri ulang pada simulasi yang menggunakan jalur hasil pengukuran, kemudian pengujian diulang. Tahap terakhir adalah analisis data yang membandingkan kinerja dan kebutuhan sumber daya perangkat keras kedua algoritma, kemudian dilanjutkan dengan penyusunan laporan tugas akhir. Penelitian ini direncanakan berlangsung selama satu semester (6 bulan). Rincian jadwal kegiatan disajikan pada Tabel 3.3.

Tabel 3.3: Jadwal pelaksanaan tugas akhir

No.	Kegiatan	Bulan 1	Bulan 2	Bulan 3	Bulan 4	Bulan 5	Bulan 6
1	Studi literatur dan penentuan spesifikasi						
2	Perancangan diagram blok dan diagram alir						
3	Implementasi RTL FXLMS dan FXNLMS dalam Verilog						
4	Penyusunan *testbench*, simulasi RTL, estimasi $\hat{S}(z)$, dan penetapan μ						
5	Sintesis, implementasi, dan evaluasi utilisasi dan *timing*						
6	Perakitan prototipe dan estimasi jalur sekunder						
7	Pemrograman FPGA dan pengujian prototipe: NR, konvergensi, dan *steady-state error*						
8	Analisis data: kinerja dan kebutuhan sumber daya perangkat keras						
9	Penyusunan laporan tugas akhir						

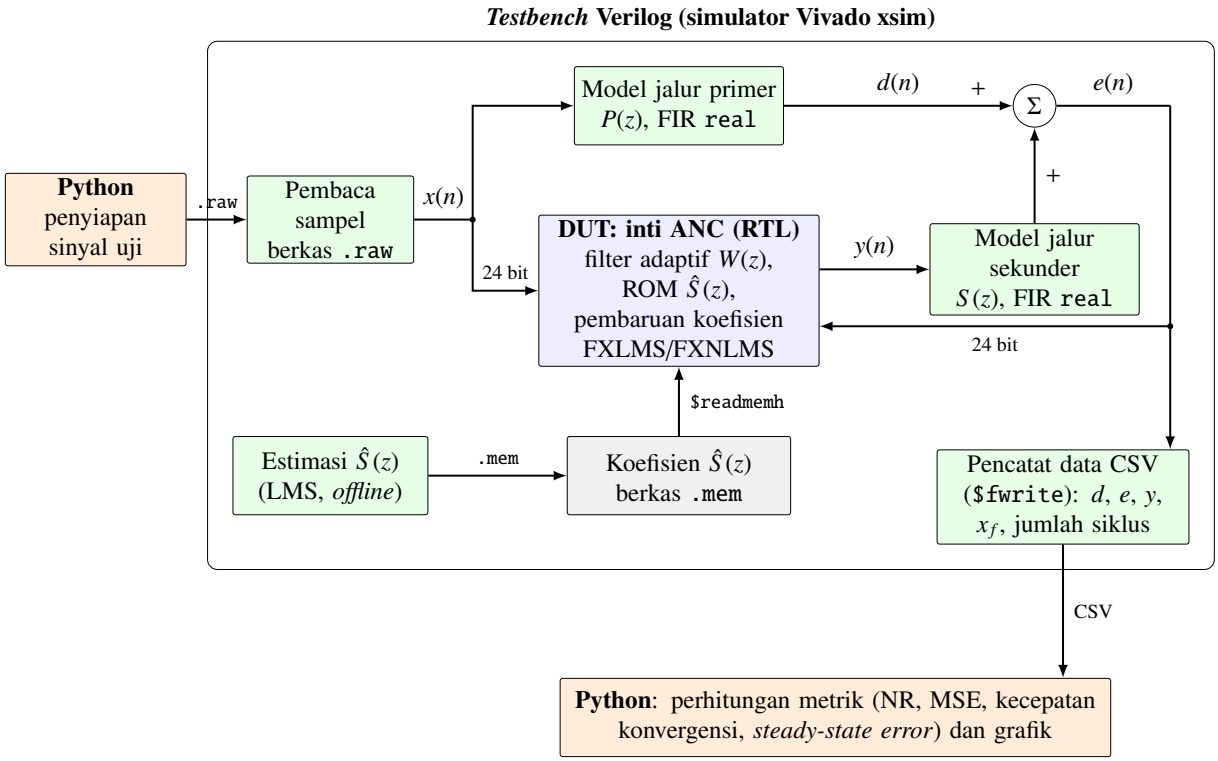
3.4 Perancangan Sistem ANC *Headphone* Berbasis FPGA

Bagian ini menjelaskan rancangan sistem ANC *headphone* yang menjadi objek implementasi FXLMS dan FXNLMS. Pembahasan mencakup arsitektur sistem, parameter rancangan, model jalur akustik dan penetapan *step-size*, estimasi jalur sekunder, implementasi dan perakitan kedua algoritma, serta arsitektur perangkat keras pada FPGA.

3.4.1 Arsitektur Sistem

Sistem dirancang sebagai ANC *feedforward* SISO untuk satu sisi *earcup headphone*. Algoritma kontrol dirancang sebagai inti ANC berbentuk RTL Verilog dan diimplementasikan pada FPGA Artix-7 XC7A35T (*board* Arty A7-35T). Inti ANC yang sama digunakan pada dua lingkungan, yaitu lingkungan simulasi dan prototipe. Pada lingkungan simulasi, inti ANC menjadi rancangan yang diuji (*design under test*, DUT) di dalam *testbench*. Pada prototipe, inti ANC dibungkus dengan blok antarmuka mikrofon, DAC, dan pencatat data. Karena inti ANC tidak diubah ketika dipindahkan ke prototipe, hasil verifikasi pada simulasi tetap berlaku pada perangkat keras.

Diagram blok lingkungan simulasi ditunjukkan pada Gambar 3.2. Lingkungan akustik *headphone* digantikan oleh *testbench* Verilog yang memodelkan sumber kebisingan, jalur primer, jalur sekunder, dan titik penjumlahan pada mikrofon *error*.

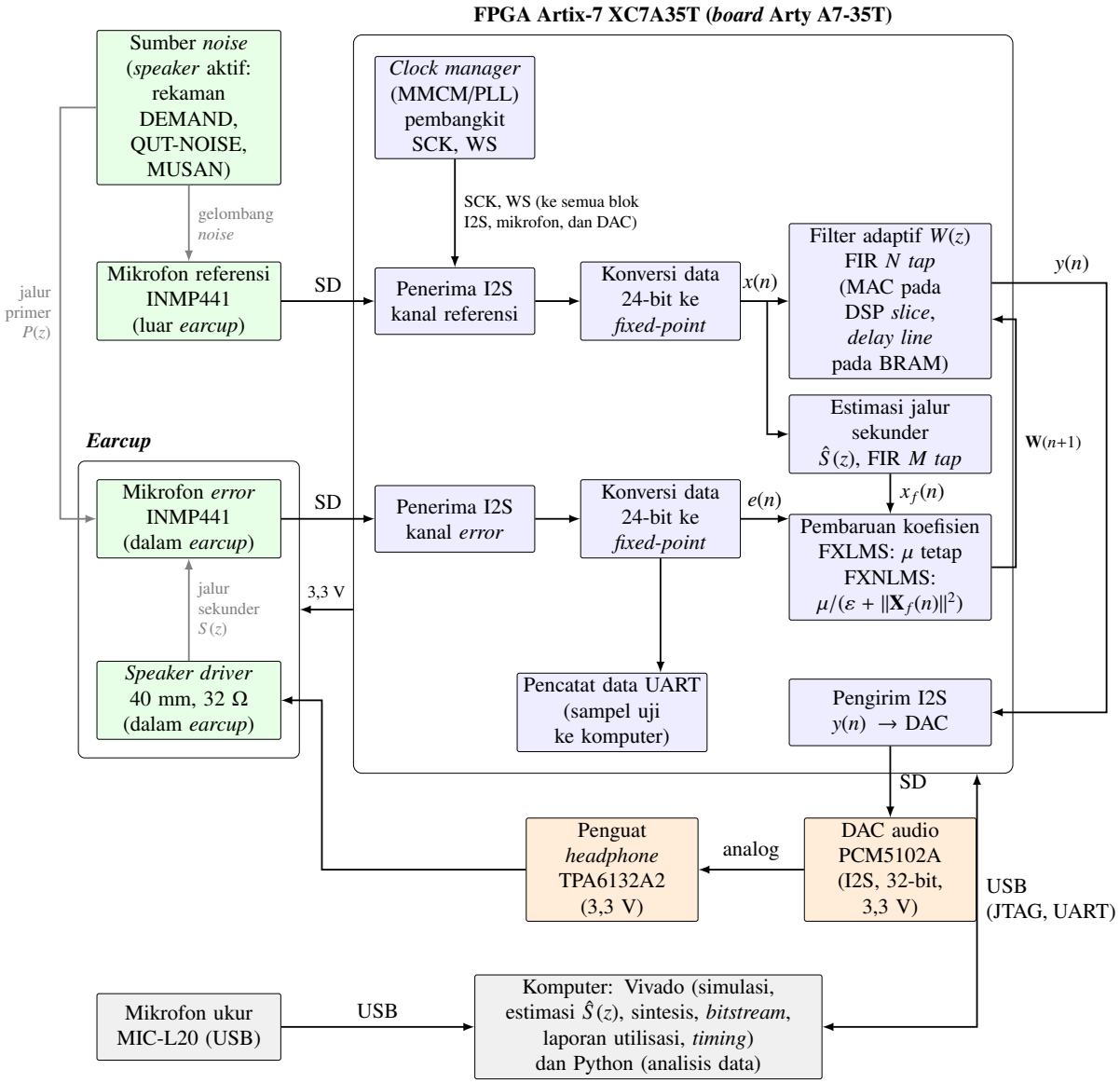


Gambar 3.2: Diagram blok sistem simulasi ANC *headphone*

Pembaca sampel membaca berkas `.raw` hasil penyiapan sinyal satu sampel setiap periode sampel, lalu mengubahnya menjadi sinyal referensi $x(n)$. Sinyal $x(n)$ dilewatkan ke model jalur primer $P(z)$ untuk menghasilkan *noise* primer $d(n)$ pada posisi mikrofon *error*. Sinyal $x(n)$ yang sama dikuantisasi ke format 24 bit dan diberikan ke DUT. DUT menghasilkan sinyal *anti-noise* $y(n)$, yang dilewatkan ke model jalur sekunder $S(z)$. Keluaran $S(z)$ dijumlahkan dengan $d(n)$ sehingga diperoleh sinyal residual $e(n)$ sesuai Persamaan (2.4). Sinyal $e(n)$ dikuantisasi ke format 24 bit dan diberikan kembali ke DUT sebagai sinyal *error*. Dengan demikian, DUT bekerja dalam lingkar umpan balik yang sama dengan prototipe.

Sebelum DUT dijalankan, blok estimasi pada *testbench* mengestimasi $\hat{S}(z)$ secara *offline* dengan prosedur pada Subbab 3.4.4, lalu menuliskan koefisiennya ke berkas `.mem`. Berkas tersebut dimuat ke ROM di dalam DUT menggunakan `$readmemh`. Pencatat data menyimpan $d(n)$, $e(n)$, $y(n)$, $x_f(n)$, dan jumlah siklus *clock* setiap iterasi ke berkas CSV menggunakan `$fwrite`. Berkas CSV tersebut diolah oleh Python untuk menghitung metrik dan menampilkan grafik.

Diagram blok prototipe ditunjukkan pada Gambar 3.3. Diagram tersebut terbagi menjadi empat bagian, yaitu domain akustik, FPGA, rantai analog keluaran, serta perangkat pendukung.



Gambar 3.3: Diagram blok prototipe ANC *headphone* berbasis FPGA

Pada domain akustik, sumber *noise* berupa *speaker* aktif memutar rekaman bising dari repositori pada Tabel 3.2. Gelombang *noise* ditangkap oleh mikrofon referensi di luar *earcup* sebagai sinyal referensi. Gelombang yang sama merambat melalui jalur primer $P(z)$ menuju mikrofon *error* di dalam *earcup*. Mikrofon *error* menangkap sinyal residual, yaitu gabungan *noise* dari jalur primer dan *anti-noise* dari *speaker driver* yang merambat melalui jalur sekunder $S(z)$. Mikrofon referensi ditempatkan di antara sumber *noise* dan *earcup* pada posisi tetap di dudukan uji. Jarak mikrofon referensi terhadap *earcup* menentukan *delay* jalur primer, sehingga jarak tersebut dicatat dan dievaluasi terhadap kendala kausalitas pada Subbab 3.5.4. Kedua mikrofon dicatu langsung oleh jalur 3,3 V *board* Arty A7-35T melalui pin VCC Pmod.

Pada FPGA, *clock manager* membangkitkan sinyal SCK dan WS sehingga FPGA berperan sebagai *master* bagi kedua mikrofon dan DAC. Data serial (SD) dari kedua mikrofon diterima oleh blok penerima I2S, kemudian dikonversi dari format 24-bit menjadi format *fixed-point* internal. Hasil konversi tersebut adalah sinyal referensi $x(n)$ dan sinyal *error* $e(n)$ yang diberikan ke inti ANC. Inti ANC menghitung $y(n)$, $x_f(n)$, dan koefisien baru $\mathbf{W}(n+1)$ dengan cara

yang sama seperti pada simulasi. Blok pencatat data mengirimkan sampel sinyal ke komputer melalui UART sehingga kurva pembelajaran dan besar peredaman dapat dihitung dari sinyal yang benar-benar diproses oleh perangkat keras.

Sinyal $y(n)$ dikirim melalui blok pengirim I2S ke DAC PCM5102A. Sinyal analog keluaran DAC dikuatkan oleh penguat TPA6132A2 untuk menggerakkan *speaker driver* di dalam *earcup*. DAC dan penguat juga dicatu oleh jalur 3,3 V *board* Arty A7-35T, sedangkan *speaker driver* bersifat pasif dan memperoleh daya dari keluaran penguat. Komputer terhubung ke *board* melalui satu kabel USB yang membawa jalur JTAG untuk pemrograman FPGA dan jalur UART untuk pencatatan data. Tingkat peredaman pada prototipe juga diukur secara independen menggunakan mikrofon ukur MIC-L20 yang terhubung ke komputer.

3.4.2 Parameter Rancangan

Parameter rancangan berlaku sama untuk FXLMS dan FXNLMS, baik pada simulasi maupun pada prototipe. Frekuensi *sampling* f_s ditetapkan sebesar 48 kHz (periode sampel 20,83 μs), sama dengan format rekaman DEMAND dan QUT-NOISE. Frekuensi *clock* sistem f_{clk} ditetapkan sebesar 100 MHz sesuai *clock board* Arty A7-35T, sehingga anggaran siklus untuk menyelesaikan satu iterasi algoritma adalah $f_{clk}/f_s \approx 2083$ siklus per sampel. Panjang filter adaptif ditetapkan $N = 64$ *tap* dan panjang estimasi jalur sekunder ditetapkan $M = 32$ *tap*. Nilai M dipilih agar mencakup respons impuls jalur sekunder, baik pada model maupun hasil estimasi pada prototipe. Kedua nilai tersebut dibuat tetap agar perbandingan kedua algoritma tidak dipengaruhi oleh perbedaan panjang filter.

Lebar bit mengikuti lebar port blok DSP48E1. Sinyal $x(n)$, $e(n)$, dan $x_f(n)$ menggunakan format 24 bit *signed*; pada prototipe, nilai ini diambil dari 24 bit teratas data mikrofon 24-bit. Koefisien $\mathbf{W}$ dan $\hat{\mathbf{s}}$ menggunakan 25 bit pada masukan pengali, sedangkan $\mathbf{W}$ disimpan dengan bit pecahan tambahan agar pembaruan yang sangat kecil tidak hilang akibat pembulatan. Hasil MAC diakumulasikan pada akumulator 48 bit. Lebar bit tersebut menjadi nilai awal dan ditambah apabila simulasi RTL pada Subbab 3.5.1 menunjukkan *overflow* atau kegagalan konvergensi akibat galat kuantisasi. *Step-size* μ kedua algoritma dipilih dalam bentuk 2^{-k} agar perkalian dengan μ dapat diganti dengan pergeseran bit, dengan nilai k ditetapkan melalui penelusuran pada simulasi RTL (Subbab 3.4.3). Konstanta ε pada FXNLMS ditetapkan sebagai konstanta *fixed-point* positif kecil yang mencegah pembagian dengan nol. Untuk pengujian, kurva pembelajaran dirata-ratakan dari $R = 10$ pengulangan, sedangkan *steady-state error* dan NR dihitung pada jendela kondisi tunak sepanjang $K = 48.000$ sampel (1 s).

3.4.3 Model Jalur Akustik dan Penetapan *Step-Size*

Model jalur akustik disusun di dalam *testbench* Verilog menggunakan bilangan `real`. Model mengikuti struktur ANC *feedforward* pada Bab 2, yaitu jalur primer $P(z)$ dan jalur sekunder $S(z)$ yang dimodelkan sebagai filter FIR, sehingga sinyal residual pada mikrofon *error* mengikuti Persamaan (2.4). Koefisien FIR $P(z)$ dan $S(z)$ dibaca oleh *testbench* dari berkas teks.

Sebelum prototipe tersedia, koefisien FIR $P(z)$ dan $S(z)$ diambil dari respons impuls jalur primer dan jalur sekunder *headphone* yang telah dipublikasikan, atau berupa model sintetis FIR dengan *delay* murni dan respons *band-pass* pada pita 100–1000 Hz dengan asumsi yang dinyatakan secara eksplisit. Setelah prototipe dirakit, model tersebut diganti dengan respons impuls jalur primer dan jalur sekunder hasil pengukuran pada prototipe dengan prosedur pada Subbab 3.4.4, kemudian simulasi RTL diulang. Dengan cara ini, hasil prototipe dapat diband-

ingkan dengan hasil simulasi pada jalur akustik yang sama. Model $S(z)$ dibuat memiliki *delay* minimal satu sampel, yang mewakili *delay* konversi digital-analog dan analog-digital, sehingga $e(n)$ hanya bergantung pada $y(n-1)$ dan sampel sebelumnya.

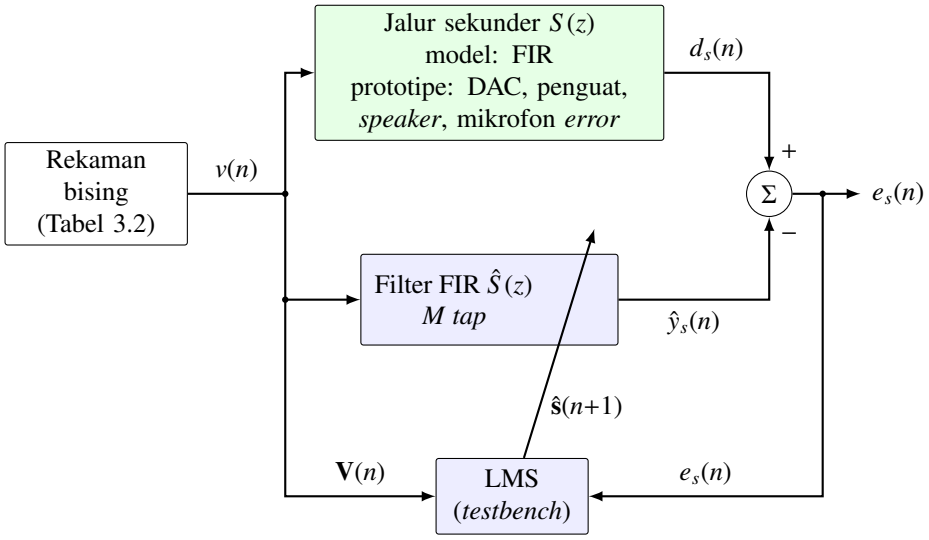
Step-size kedua algoritma ditetapkan langsung pada simulasi RTL. Nilai *step-size* FXLMS dan FXNLMS masing-masing ditentukan melalui penelusuran beberapa nilai 2^{-k} (*tuning*) pada sinyal uji yang sama. Cara ini dipilih agar kedua algoritma dibandingkan pada kondisi terbaiknya, bukan pada nilai yang kebetulan merugikan salah satu algoritma. Karena nilai μ berbentuk pergeseran bit, nilai tersebut dijadikan parameter rancangan sehingga penelusuran tidak memerlukan perubahan RTL. Dengan $N = 64$ dan $M = 32$, satu iterasi membutuhkan beberapa ratus siklus *clock*, sehingga penelusuran pada simulasi RTL masih dapat dilakukan dalam waktu yang wajar.

Kedua algoritma diuji pada kondisi yang sama, yaitu sinyal uji, jalur primer dan sekunder, panjang filter, frekuensi *sampling*, lebar bit, dan kondisi awal koefisien filter $\mathbf{W}(0) = \mathbf{0}$.

3.4.4 Estimasi Jalur Sekunder

FXLMS dan FXNLMS sama-sama membutuhkan estimasi jalur sekunder $\hat{S}(z)$ untuk menghasilkan sinyal referensi terfilter $x_f(n)$ pada Persamaan (2.11). Oleh karena itu, jalur sekunder diestimasi terlebih dahulu sebelum sistem ANC dijalankan (*offline*). Estimasi dilakukan dua kali dengan skema yang sama, yaitu terhadap model $S(z)$ dan terhadap jalur sekunder nyata pada prototipe. Pada kedua estimasi, perhitungan LMS dilakukan oleh blok estimasi pada *testbench* Verilog menggunakan bilangan `real`. Pada kedua estimasi, filter adaptif $W(z)$ belum dijalankan.

Skema estimasi jalur sekunder ditunjukkan pada Gambar 3.4. Sinyal identifikasi $v(n)$ berupa potongan rekaman bising dari repositori pada Tabel 3.2 diberikan ke jalur sekunder. Pada estimasi pertama, jalur sekunder berupa model FIR $S(z)$. Pada prototipe, jalur sekunder berupa rantai keluaran yang sama dengan sinyal *anti-noise*, yaitu DAC, penguat, *speaker driver*, dan mikrofon *error*. Sinyal keluaran jalur sekunder digunakan sebagai sinyal yang diinginkan $d_s(n)$, sedangkan sinyal $v(n)$ yang sama digunakan sebagai masukan filter FIR $\hat{S}(z)$ sepanjang *M tap*.



Gambar 3.4: Skema estimasi jalur sekunder secara *offline*

Keluaran $\hat{S}(z)$ dikurangkan dari $d_s(n)$ sehingga diperoleh galat estimasi $e_s(n)$. Galat $e_s(n)$ dan vektor masukan $\mathbf{V}(n) = [v(n), v(n-1), \ldots, v(n-M+1)]^T$ diberikan ke blok LMS. Blok LMS menghitung koefisien baru $\hat{\mathbf{s}}(n+1)$ mengikuti Persamaan (2.9):

$$\hat{\mathbf{s}}(n+1) = \hat{\mathbf{s}}(n) + 2\mu_s \, e_s(n) \, \mathbf{V}(n) \tag{3.1}$$

dengan $\hat{\mathbf{s}}(n) = [\hat{s}_0(n), \hat{s}_1(n), \ldots, \hat{s}_{M-1}(n)]^T$ sebagai vektor koefisien $\hat{S}(z)$ dan μ_s sebagai *step-size* estimasi. Pada Persamaan (3.1), $e_s(n)$ berperan sebagai sinyal galat dan $\mathbf{V}(n)$ berperan sebagai vektor masukan pada LMS. Koefisien baru tersebut dimuat kembali ke filter FIR $\hat{S}(z)$, yang ditunjukkan oleh panah diagonal dari blok LMS yang menembus blok $\hat{S}(z)$ pada Gambar 3.4. Proses ini diulang hingga $e_s(n)$ mencapai kondisi tunak.

Pada prototipe, pengambilan data dilakukan menggunakan *bitstream* identifikasi yang terpisah dari *bitstream* ANC. *Bitstream* ini memutar $v(n)$ yang disimpan pada BRAM ke DAC secara berulang dan secara bersamaan mengirimkan sampel mikrofon *error* ke komputer melalui UART. Karena pemutaran dan pencuplikan dikendalikan oleh *clock* yang sama, $v(n)$ dan $d_s(n)$ tersinkron per sampel. Pengukuran dilakukan tanpa sumber *noise* eksternal agar sinyal pada mikrofon *error* hanya berasal dari *speaker*. Rekaman $d_s(n)$ kemudian dibaca oleh *testbench* menggunakan `$fread`, dan Persamaan (3.1) dihitung oleh blok estimasi yang sama dengan yang digunakan pada estimasi terhadap model. Prosedur yang sama dengan sumber *noise* eksternal aktif dan mikrofon referensi sebagai masukan digunakan untuk memperoleh respons impuls jalur primer $P(z)$ prototipe bagi simulasi RTL.

Pada kedua estimasi, koefisien $\hat{S}(z)$ yang diperoleh dikuantisasi ke format koefisien 25 bit, lalu ditulis ke berkas `.mem` dalam format heksadesimal menggunakan `$fwrite`. Berkas tersebut dimuat sebagai ROM koefisien tetap pada rancangan FXLMS dan FXNLMS, yaitu menggunakan `$readmemh` pada simulasi dan sebagai nilai awal BRAM pada *bitstream* prototipe. Dengan demikian, kedua algoritma menggunakan $\hat{S}(z)$ yang identik, sehingga perbedaan hasil kedua algoritma tidak dipengaruhi oleh kualitas estimasi jalur sekunder.

Respons impuls $\hat{S}(z)$ prototipe juga memuat *delay* total jalur sekunder, yaitu *delay* pengirim I2S, DAC, penguat, rambatan akustik, mikrofon *error*, dan penerima I2S. *Delay* tersebut terlihat sebagai koefisien awal $\hat{s}_i$ yang bernilai mendekati nol dan digunakan pada analisis kausalitas di Subbab 3.5.4.

3.4.5 Implementasi dan Perakitan FXLMS dan FXNLMS

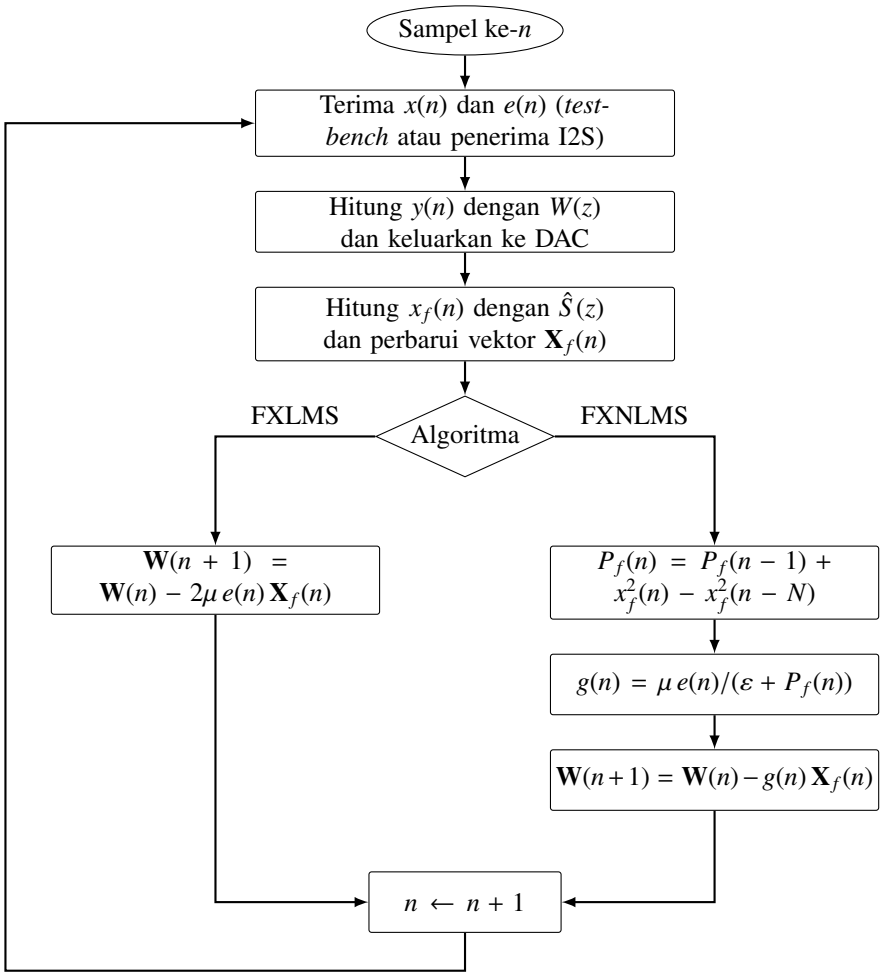
FXLMS dan FXNLMS diimplementasikan dari modul RTL yang sama. FXLMS dibangun terlebih dahulu sebagai rancangan dasar, kemudian FXNLMS diperoleh dengan mengganti unit pembaruan koefisiennya saja. Implementasi dilakukan secara bertahap dari modul terkecil, dan setiap modul diuji pada *testbench* tersendiri sebelum dirakit menjadi inti ANC. Rincian arsitektur setiap modul dijelaskan pada Subbab 3.4.6.

Tahap pertama adalah implementasi modul dasar, yaitu unit MAC yang dipetakan ke DSP48E1, *circular buffer* berbasis BRAM untuk $\mathbf{X}(n)$ dan $\mathbf{X}_f(n)$, BRAM *dual-port* untuk koefisien $\mathbf{W}$, serta ROM $\hat{S}(z)$ yang diinisialisasi dari berkas `.mem`. Setiap modul diuji dengan masukan yang diketahui, misalnya urutan baca-tulis *buffer* dan hasil perkalian-akumulasi yang dibandingkan dengan nilai hasil perhitungan manual.

Tahap kedua adalah perakitan filter $W(z)$ dan $\hat{S}(z)$ dari modul dasar tersebut. Kedua filter diuji dengan pembaruan koefisien dinonaktifkan, sesuai uji fungsional per blok pada Sub-

bab 3.5.1.

Tahap ketiga adalah implementasi unit kendali berupa FSM yang menjadwalkan pemrosesan setiap sampel sesuai Gambar 3.5. Keluaran $y(n)$ dihitung paling awal dan langsung dikeluarkan agar latensi dari masukan ke keluaran sekecil mungkin. Setelah itu, $x_f(n)$ dihitung dan dimasukkan ke *buffer* $\mathbf{X}_f(n)$, kemudian koefisien diperbarui.



Gambar 3.5: Alur pemrosesan setiap sampel pada FXLMS dan FXNLMS

Tahap keempat adalah implementasi unit pembaruan koefisien FXLMS. Karena $\mu = 2^{-k}$, faktor skalar $2\mu\, e(n)$ dihitung dengan pergeseran bit. Faktor tersebut kemudian dikalikan dengan setiap elemen $\mathbf{X}_f(n)$, dan hasilnya dituliskan kembali ke BRAM koefisien. Setelah unit ini dirakit bersama filter dan unit kendali, inti ANC FXLMS diuji dalam lingkar tertutup pada *testbench*.

Tahap kelima adalah implementasi FXNLMS dengan mengganti unit pembaruan koefisien FXLMS, tanpa mengubah modul lain. Unit pembaruan FXNLMS ditambah register energi $P_f(n) = \|\mathbf{X}_f(n)\|^2$ yang diperbarui secara rekursif:

$$P_f(n) = P_f(n-1) + x_f^2(n) - x_f^2(n-N) \tag{3.2}$$

dengan $P_f(0) = 0$. Bentuk ini hanya membutuhkan satu pengali kuadrat tambahan, karena

$x_f(n-N)$ sudah tersedia pada *buffer* $\mathbf{X}_f(n)$ yang dibuat sepanjang $N+1$ sampel. Unit ini juga ditambah pembagi sekuensial yang menghitung faktor skalar satu kali per sampel:

$$g(n) = \frac{\mu\, e(n)}{\varepsilon + P_f(n)} \tag{3.3}$$

Setelah $g(n)$ diperoleh, N koefisien diperbarui melalui jalur yang sama dengan FXLMS, yaitu $\mathbf{W}(n+1) = \mathbf{W}(n) - g(n)\,\mathbf{X}_f(n)$, yang setara dengan Persamaan (2.14). Dengan cara ini, tambahan perangkat keras FXNLMS terbatas pada register energi, satu pengali kuadrat, dan satu pembagi.

Tahap keenam adalah integrasi inti ANC dengan blok antarmuka prototipe, yaitu *clock manager*, penerima I2S, konversi data, pengirim I2S, dan pencatat data UART. Blok antarmuka tersebut diuji secara terpisah pada simulasi sebelum digabungkan. Hasil integrasi disintesis menjadi satu *bitstream* untuk setiap algoritma, dengan koefisien $\hat{S}(z)$ prototipe dimuat sebagai nilai awal BRAM.

Perakitan prototipe dilakukan bersamaan dengan implementasi RTL. Kedua mikrofon INMP441, DAC PCM5102A, dan penguat TPA6132A2 dihubungkan ke *board* Arty A7-35T melalui konektor Pmod dan dicatu oleh jalur 3,3 V *board*. Kedua mikrofon menerima sinyal SCK dan WS yang sama dari FPGA, sedangkan jalur data SD masing-masing mikrofon dihubungkan ke pin yang terpisah sesuai kedua penerima I2S pada Gambar 3.3. DAC menerima sinyal BCK, LRCK, dan DIN dari pengirim I2S. Keluaran analog DAC dihubungkan ke masukan penguat, dan keluaran penguat dihubungkan ke *speaker driver*.

Speaker driver dan mikrofon *error* dipasang di dalam *earcup*, dengan mikrofon *error* berada di depan *speaker* pada posisi yang mewakili telinga. Mikrofon referensi dipasang di luar *earcup*, di antara sumber *noise* dan *earcup*. Posisi *earcup*, kedua mikrofon, dan sumber *noise* dikunci pada dudukan uji, dan jarak antarkomponen dicatat agar kondisi pengukuran sama pada setiap pengujian. Sebelum ANC dijalankan, setiap kanal diperiksa terlebih dahulu. Data kedua mikrofon diperiksa melalui UART ketika sumber *noise* memutar rekaman, sedangkan rantai keluaran diperiksa menggunakan *bitstream* identifikasi pada Subbab 3.4.4.

3.4.6 Arsitektur Perangkat Keras pada FPGA

Kedua algoritma dirancang dalam Verilog dengan aritmetika *fixed-point*, kemudian disintesis dan diimplementasikan menggunakan Vivado pada *board* Arty A7-35T. Rancangan terdiri atas inti ANC dan blok antarmuka. Inti ANC memiliki antarmuka paralel sederhana, yaitu masukan $x(n)$ dan $e(n)$ 24 bit, sinyal penanda sampel baru, keluaran $y(n)$, sinyal penanda selesai, serta *reset*. Pada simulasi, antarmuka ini dihubungkan langsung dengan *testbench*. Pada prototipe, antarmuka ini dihubungkan dengan blok *clock manager*, penerima I2S, konversi data, pengirim I2S, dan pencatat data UART sesuai Gambar 3.3. Inti ANC terdiri atas unit kendali, unit MAC, memori, dan unit pembaruan koefisien.

Unit kendali berupa *finite state machine* (FSM) yang dipicu satu kali setiap sampel baru tersedia, baik dari *testbench* maupun dari penerima I2S. FSM menjadwalkan urutan operasi pada Gambar 3.5, yaitu perhitungan $y(n)$ sebanyak N operasi MAC, perhitungan $x_f(n)$ sebanyak M operasi MAC, lalu pembaruan N koefisien. Karena anggaran siklus per sampel jauh lebih besar daripada jumlah operasi MAC per iterasi, operasi MAC dijalankan secara berurutan (*time-multiplexed*) pada sejumlah kecil unit MAC yang dipetakan ke DSP48E1. Jumlah unit MAC

dibuat sama pada kedua rancangan. Seluruh operasi satu iterasi harus selesai sebelum sampel berikutnya tiba, yaitu dalam kurang dari 2083 siklus *clock*.

Delay line sinyal referensi $\mathbf{X}(n)$ dan sinyal referensi terfilter $\mathbf{X}_f(n)$ disimpan pada BRAM sebagai *circular buffer*. *Buffer* $\mathbf{X}_f(n)$ dibuat sepanjang $N + 1$ sampel agar $x_f(n - N)$ pada Persamaan (3.2) tetap tersedia. Koefisien $\mathbf{W}(n)$ disimpan pada BRAM *dual-port* sehingga pembacaan dan penulisan koefisien hasil pembaruan dapat dilakukan pada siklus yang sama. Koefisien $\hat{S}(z)$ disimpan pada ROM berbasis BRAM yang diinisialisasi dari berkas `.mem`.

Unit pembaruan koefisien menjadi satu-satunya blok yang berbeda antara kedua rancangan. Pada FXLMS, unit ini menghitung faktor $2\mu\, e(n)$, lalu mengalikannya dengan setiap elemen $\mathbf{X}_f(n)$. Pada FXNLMS, unit ini ditambah dengan register energi $P_f(n)$ beserta pengali kuadrat untuk Persamaan (3.2), serta unit pembagi sekuensial untuk menghitung $g(n)$ pada Persamaan (3.3). Unit pembagi dibangun menggunakan IP *Divider Generator* Vivado atau pembagi iteratif *radix*-2. Pembagi sekuensial dipilih karena pembagian hanya dilakukan satu kali per sampel, sehingga latensi pembagian yang lebih panjang masih berada dalam anggaran siklus, sedangkan sumber daya logika yang dibutuhkan lebih kecil dibandingkan pembagi paralel.

Panjang filter N dan M, lebar bit, jumlah unit MAC, frekuensi *clock*, dan blok antarmuka dibuat sama pada kedua rancangan. Setiap algoritma menghasilkan satu *bitstream* tersendiri. Dengan demikian, perbedaan utilisasi sumber daya dan latensi hanya berasal dari perbedaan unit pembaruan koefisien. Koefisien $\mathbf{W}$ dapat dikembalikan ke nol melalui sinyal *reset*, yang pada prototipe dihubungkan dengan tombol pada *board*, agar setiap pengujian dimulai dari kondisi awal yang sama.

3.5 Metode Pengujian dan Analisis Data

Pengujian dilakukan untuk menjawab keempat rumusan masalah. Seluruh metrik dihitung dengan prosedur yang sama untuk FXLMS dan FXNLMS agar hasilnya dapat dibandingkan secara langsung. Pembahasan mencakup verifikasi fungsional rancangan RTL, kinerja peredaman, kecepatan konvergensi dan *steady-state error*, kebutuhan sumber daya perangkat keras, serta analisis data akhir.

Metrik pada simulasi RTL dan prototipe dihitung oleh skrip Python yang sama. Pada simulasi, skrip tersebut membaca berkas CSV keluaran *testbench*. Pada prototipe, skrip tersebut membaca berkas data yang dicatat melalui UART. Dengan demikian, perbedaan hasil keduanya tidak berasal dari perbedaan cara perhitungan. Simulasi dijalankan secara *batch* menggunakan skrip Tcl. Pada simulasi, *testbench* memberikan sampel berikutnya segera setelah DUT menyelesaikan satu iterasi, tanpa menunggu 2083 siklus, sehingga waktu simulasi lebih singkat tanpa mengubah hasil adaptasi.

3.5.1 Verifikasi Fungsional Rancangan RTL

Rancangan RTL diverifikasi menggunakan *testbench* pada simulator Vivado sebelum disintesis dan dijalankan pada prototipe. Verifikasi dilakukan dalam dua tingkat. Tingkat pertama adalah uji fungsional per blok. Pada uji ini, pembaruan koefisien dinonaktifkan, $\mathbf{W}$ diisi dengan koefisien yang diketahui, dan $\hat{S}(z)$ dimuat dari berkas `.mem`. Keluaran $y(n)$ dan $x_f(n)$ dari DUT kemudian dibandingkan dengan hasil konvolusi yang dihitung oleh *testbench* dari masukan dan koefisien yang sama. Karena kedua sinyal tersebut tidak bergantung pada umpan balik, selisihnya harus berada dalam batas galat kuantisasi format 24 bit. Tingkat kedua adalah uji lingkar tertutup, yaitu DUT dijalankan dengan pembaruan koefisien aktif dalam lingkar ANC pada

Gambar 3.2.

Rancangan RTL dinyatakan lolos verifikasi jika memenuhi tiga kriteria. Pertama, selisih $y(n)$ dan $x_f(n)$ pada uji per blok tidak melebihi batas galat kuantisasi. Kedua, pada uji lingkar tertutup, FXLMS dan FXNLMS konvergen tanpa divergensi pada seluruh rekaman uji, yang ditunjukkan oleh MSE yang menurun hingga kondisi tunak. Ketiga, tidak terjadi *overflow* pada akumulator, register energi, dan koefisien, yang dipantau oleh pemeriksaan otomatis di dalam *testbench* yang menghentikan simulasi dan melaporkan lokasi *overflow*. Jika belum lolos, rancangan diagram blok atau RTL diperbaiki, atau lebar bit pada bagian yang bermasalah ditambah, sebelum dilanjutkan ke tahap sintesis.

Verifikasi ini dilakukan pada FXLMS dan FXNLMS dengan sinyal uji yang sama. Blok antarmuka I2S dan UART diverifikasi secara terpisah pada simulasi sebelum digabungkan dengan inti ANC. Hasil verifikasi ini digunakan untuk menjawab rumusan masalah pertama bersama hasil *timing* dan latensi pada Subbab 3.5.4 serta keberhasilan *bitstream* kedua algoritma bekerja dan konvergen pada prototipe.

3.5.2 Pengujian Kinerja Peredaman

Kinerja peredaman dinyatakan sebagai besar peredaman kebisingan (*noise reduction*, NR) dalam dB. NR merupakan perbandingan daya *noise* pada mikrofon *error* tanpa ANC terhadap daya sinyal residual setelah ANC bekerja:

$$\mathrm{NR} = 10 \log_{10} \left(\frac{\sum_n d^2(n)}{\sum_n e^2(n)} \right) \tag{3.4}$$

dengan $d(n)$ sebagai *noise* primer pada mikrofon *error* tanpa ANC dan $e(n)$ sebagai sinyal residual setelah ANC bekerja. Penjumlahan dilakukan pada K sampel setelah algoritma mencapai kondisi tunak.

Pada simulasi, $d(n)$ dihitung langsung oleh *testbench* dari model $P(z)$ dan dicatat bersama $e(n)$, sehingga keduanya tersedia pada simulasi yang sama. Pada prototipe, rekaman bising diputar melalui *speaker* aktif dengan tingkat volume yang sama untuk setiap pengujian. Sinyal $d(n)$ dicatat dari mikrofon *error* melalui UART ketika keluaran $y(n)$ dinonaktifkan, sedangkan sinyal $e(n)$ dicatat dengan cara yang sama ketika ANC aktif. Pengujian dilakukan untuk tiga rekaman dari setiap repositori DEMAND, QUT-NOISE, dan MUSAN. Sebagai pembanding independen, tingkat tekanan suara di dalam *earcup* juga diukur menggunakan mikrofon ukur MIC-L20 pada kondisi ANC tidak aktif dan aktif. Spektrum daya $d(n)$ dan $e(n)$ juga ditampilkan untuk melihat sebaran peredaman pada pita 100–1000 Hz. Nilai NR hasil simulasi untuk rekaman yang sama dilaporkan sebagai acuan. Hasil pengujian ini digunakan untuk menjawab rumusan masalah kedua.

3.5.3 Pengujian Kecepatan Konvergensi dan *Steady-State Error*

Perilaku konvergensi diamati melalui kurva pembelajaran (*learning curve*), yaitu nilai MSE terhadap jumlah iterasi. Sinyal $e(n)$ dicatat sejak koefisien $\mathbf{W}$ dikembalikan ke nol, yaitu dari DUT pada simulasi dan melalui UART pada prototipe. Karena hasil adaptasi bergantung pada potongan rekaman yang digunakan, setiap skenario diulang sebanyak R kali dengan potongan rekaman yang berbeda, kemudian hasilnya dirata-ratakan:

$$\mathrm{MSE}(n) = \frac{1}{R}\sum_{r=1}^{R} e_r^2(n) \tag{3.5}$$

dengan MSE(n) sebagai nilai MSE rata-rata pada iterasi ke-n dan $e_r(n)$ sebagai sinyal *error* pada iterasi ke-n dari pengulangan ke-r. Kurva MSE(n) diperhalus menggunakan rata-rata bergerak sebelum dianalisis.

Steady-state error dihitung sebagai rata-rata MSE(n) pada K iterasi terakhir setelah algoritma konvergen. Kecepatan konvergensi dinyatakan sebagai jumlah iterasi, beserta waktu dalam detik, yang dibutuhkan hingga MSE(n) masuk dan tetap berada dalam pita toleransi 1 dB di atas nilai *steady-state error* tersebut.

Pengujian dilakukan pada dua skenario. Skenario pertama menggunakan rekaman bising dengan tingkat daya tetap. Skenario kedua menggunakan rekaman bising yang tingkat dayanya dinaikkan dan diturunkan secara tiba-tiba sebesar 10 dB selama pengujian untuk memunculkan fluktuasi daya sinyal *broadband*. Perubahan tingkat daya dibuat pada berkas rekaman sebelum digunakan, sehingga waktu perubahannya diketahui. Skenario kedua dipilih karena *step-size* tetap pada FXLMS dirancang untuk satu tingkat daya sinyal referensi.

Pada kedua skenario, FXLMS diuji dengan beberapa nilai *step-size* hasil penelusuran pada simulasi RTL untuk menunjukkan hubungan antara kecepatan konvergensi dan kestabilan. FXNLMS diuji dengan *step-size* terbaik hasil penelusuran yang sama. Perbandingan dilakukan dengan dua cara: kecepatan konvergensi dibandingkan pada *steady-state error* yang setara, dan *steady-state error* dibandingkan pada kecepatan konvergensi yang setara. Kurva pembelajaran kedua algoritma juga dibandingkan sebelum dan sesudah perubahan daya. Hasil pengujian ini digunakan untuk menjawab rumusan masalah ketiga.

3.5.4 Evaluasi Kebutuhan Sumber Daya Perangkat Keras

Evaluasi ini dilakukan untuk menjawab rumusan masalah keempat. Hasil *timing* dan latensi pada evaluasi ini juga digunakan untuk menjawab rumusan masalah pertama. Kebutuhan sumber daya perangkat keras dianalisis dari tiga sisi. Pertama, jumlah operasi perkalian, penjumlahan, dan pembagian per iterasi kedua algoritma dibandingkan secara analitis berdasarkan Persamaan (2.5), (2.11), (2.13), (3.2), dan (3.3). Kedua, utilisasi sumber daya hasil implementasi Vivado dibandingkan untuk setiap jenis sumber daya:

$$U = \frac{N_{\text{terpakai}}}{N_{\text{tersedia}}} \times 100\% \tag{3.6}$$

dengan U sebagai persentase utilisasi suatu jenis sumber daya (LUT, *flip-flop*, BRAM, atau DSP *slice*), N_{terpakai} sebagai jumlah sumber daya yang digunakan rancangan menurut laporan Vivado, dan N_{tersedia} sebagai jumlah sumber daya tersebut pada FPGA. Laporan sintesis juga diperiksa untuk memastikan seluruh pengali dipetakan ke DSP48E1 dan tidak dibangun dari LUT, agar perbandingan utilisasi kedua rancangan setara. Utilisasi dilaporkan untuk keseluruhan rancangan prototipe, untuk inti ANC, dan untuk unit pembaruan koefisien secara terpisah menggunakan laporan utilisasi hierarkis, sehingga blok antarmuka yang identik pada kedua rancangan tidak menutupi selisih yang berasal dari algoritma.

Ketiga, latensi pemrosesan dan *timing* dievaluasi. Latensi pemrosesan dihitung dari jumlah

siklus *clock* sejak sampel $x(n)$ tersedia hingga $y(n)$ siap dikirim ke DAC:

$$\tau_{\text{proses}} = \frac{C_{\text{proses}}}{f_{clk}} \tag{3.7}$$

dengan τ_{proses} sebagai latensi pemrosesan, C_{proses} sebagai jumlah siklus yang diukur pada simulasi *testbench*, dan f_{clk} sebagai frekuensi *clock* sistem. Karena unit pembagi FXNLMS berada pada tahap pembaruan koefisien setelah $y(n)$ dihitung, jumlah siklus total satu iterasi juga dicatat secara terpisah dan dibandingkan dengan anggaran 2083 siklus per sampel. *Timing* dinyatakan terpenuhi jika *worst negative slack* (WNS) pada laporan Vivado bernilai tidak negatif pada frekuensi *clock* 100 MHz.

Latensi tersebut kemudian digunakan untuk memeriksa kendala kausalitas pada Subbab 2.2.1. *Delay* jalur primer ditentukan dari respons impuls $P(z)$ hasil pengukuran pada prototipe. *Delay* total jalur sekunder ditentukan dari respons impuls $\hat{S}(z)$ prototipe pada Subbab 3.4.4 ditambah τ_{proses}. Kedua nilai tersebut dibandingkan untuk menilai apakah kendala kausalitas terpenuhi pada prototipe. Selisih jumlah operasi, utilisasi, dan latensi antara FXNLMS dan FXLMS digunakan untuk menjawab rumusan masalah keempat.

3.5.5 Analisis Data Akhir

Analisis data akhir menggabungkan hasil Subbab 3.5.1, 3.5.2, 3.5.3, dan 3.5.4. Kedua algoritma dibangun dengan arsitektur, filter adaptif, estimasi jalur sekunder, lebar bit, sinyal uji, dan parameter yang identik, sehingga satu-satunya perbedaan terdapat pada unit pembaruan koefisien. Oleh karena itu, selisih NR, kecepatan konvergensi, dan *steady-state error* antara FXNLMS dan FXLMS dinyatakan sebagai hasil dari mekanisme pembaruan koefisien tersebut. Selisih utilisasi sumber daya dan latensi dinyatakan sebagai konsekuensi perangkat keras dari mekanisme yang sama.

Hasil pada skenario daya tetap dan skenario daya berfluktuasi dibandingkan untuk melihat perubahan keunggulan FXNLMS ketika daya sinyal referensi berubah. Selisih kinerja kedua algoritma kemudian disandingkan dengan selisih kebutuhan sumber daya perangkat keras, yaitu jumlah operasi per iterasi, utilisasi sumber daya FPGA, dan latensi pemrosesan per sampel. Perbandingan ini digunakan untuk menilai apakah keunggulan FXNLMS sepadan dengan tambahan sumber daya perangkat keras yang dibutuhkannya pada sistem ANC *headphone* berbasis FPGA.

Karena kedua implementasi menggunakan aritmetika *fixed-point*, hasil perbandingan tersebut sekaligus menunjukkan apakah keunggulan FXNLMS tetap muncul setelah diwujudkan dalam aritmetika *fixed-point*, sebagaimana dipertanyakan pada latar belakang. Sebagai analisis pendukung, hasil pada prototipe juga dibandingkan dengan hasil simulasi RTL yang menggunakan jalur primer dan sekunder hasil pengukuran. Selisih keduanya menunjukkan pengaruh kondisi akustik nyata, seperti derau mikrofon dan ketidaklinieran rantai analog.

DAFTAR PUSTAKA

[1] T.-Y. Chen, J.-H. Yang, C.-L. Lai, and C.-T. Wei, "Low-frequency active noise control system based on feedback FXLMS," *Electronics*, vol. 14, no. 7, 2025, Art. no. 1442. DOI: 10.3390/electronics14071442

[2] Y. Han and R. Chen, "Active noise control system based on hybrid fixed filter method and FxLMS algorithm," in *2023 7th International Conference on Electrical, Mechanical and Computer Engineering (ICEMCE)*, Xi'an, China, 2023, pp. 660–664.

[3] X. H. Xie, L. N. Zhou, and Y. J. Xie, "Design and simulation of active noise cancelling earphone system based on FXLMS algorithm," in *2022 4th International Conference on Natural Language Processing (ICNLP)*, Xi'an, China, 2022, pp. 626–630. [Online]. Available: https://ieeexplore.ieee.org/document/9885857

[4] A. Cesur, A. E. Canbey, H. Karsiyaka, and O. F. Maden, "Normalized FxLMS algorithm focused active noise cancellation (ANC) system approach in automotive industry," in *2025 16th International Conference on Electrical and Electronics Engineering (ELECO)*, Bursa, Türkiye, 2025, pp. 1–6. DOI: 10.1109/ELECO69582.2025.11329311

[5] L. Zhang and X. Qiu, "Causality study on a feedforward active noise control headset with different noise coming directions in free field," *Applied Acoustics*, vol. 80, pp. 36–44, 2014. DOI: 10.1016/j.apacoust.2014.01.004

[6] D. Shi, B. Lam, W.-S. Gan, J. Cheer, and S. J. Elliott, *Active noise control in the new century: The role and prospect of signal processing*, 2023. arXiv: 2306.01425. [Online]. Available: https://arxiv.org/abs/2306.01425

[7] A. G. Klemd, "Hardware acceleration of active noise control algorithms," Dissertation (Dr.-Ing.) Helmut-Schmidt-Universität / Universität der Bundeswehr Hamburg, Hamburg, Germany, 2023. DOI: 10.24405/15013

[8] Z. Li, Y. Yu, H. He, Y. Zhu, and R. C. de Lamare, *Study of switched step-size based filtered-x NLMS algorithm for active noise cancellation*, 2026. arXiv: 2601.16382. [Online]. Available: https://arxiv.org/abs/2601.16382

[9] A. Boutros and V. Betz, "FPGA architecture: Principles and progression," *IEEE Circuits and Systems Magazine*, vol. 21, no. 2, pp. 4–29, 2021. DOI: 10.1109/MCAS.2021.3071607

[10] E. Voltolini et al., "Design of an active noise reduction system for a cogeneration plant," in *2023 Immersive and 3D Audio: from Architecture to Automotive (I3DA)*, Bologna, Italy, 2023, pp. 1–7. DOI: 10.1109/I3DA57090.2023.10289192

[11] R. Zhang, "Investigation and evaluation of adaptive algorithms for multichannel active noise control system," M.Sc. dissertation, School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, 2023. arXiv: 2308.15485. [Online]. Available: https://arxiv.org/abs/2308.15485

[12] K. Sarkar, A. Lu, M. Mittal, Y. Zhuang, R. M. Corey, and A. C. Singer, "Latent FxLMS: Accelerating active noise control with neural adaptive filters," in *Proceedings of Forum Acusticum / EuroNoise 2025*, Málaga, Spain, 2025. DOI: 10.61782/fa.2025.0565 arXiv: 2507.03854.

[13] A. Klemd, B. Klauer, J. Timmermann, and D. Sachau, "A flexible multi-channel feedback FxLMS architecture for FPGA platforms," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, Dublin, Ireland, 2021, pp. 319–326. DOI: 10.1109/FPL53798.2021.00063

[14] Y. Pang, S. Ou, Z. Cai, and Y. Gao, "Posterior error energy minimization based combined FxNLMS and FxLMS algorithm for active noise control," *IEEE Access*, vol. 12, pp. 30 754–30 764, 2024. DOI: 10.1109/ACCESS.2024.3368860

[15] M. Arezki, A. Namane, and A. Benallal, "Performance analysis of the fast-NLMS type algorithm," in *Proceedings of the World Congress on Engineering 2017 (WCE 2017)*, vol. I, London, U.K., 2017, pp. 460–463.

[16] Xilinx, *7 series FPGAs data sheet: Overview (DS180), v2.6.1*, September 8, 2020, Xilinx, Inc., 2020.

[17] J. Thiemann, N. Ito, and E. Vincent, *DEMAND: A collection of multi-channel recordings of acoustic noise in diverse environments*, version 1.0, Zenodo, 2013. DOI: 10.5281/zenodo.1227121 [Online]. Available: https://zenodo.org/records/1227121

[18] D. Dean, S. Sridharan, R. Vogt, and M. Mason, "The QUT-NOISE-TIMIT corpus for the evaluation of voice activity detection algorithms," in *Proceedings of INTERSPEECH 2010*, Makuhari, Japan, 2010, pp. 3110–3113. DOI: 10.21437/Interspeech.2010-774

[19] D. Snyder, G. Chen, and D. Povey, *MUSAN: A music, speech, and noise corpus*, 2015. arXiv: 1510.08484. [Online]. Available: https://arxiv.org/abs/1510.08484